EXPERT INSIGHT

In color

Generative AI with LangChain

Build production-ready LLM applications and advanced agents using Python, LangChain, and LangGraph

Second Edition

1-year code support

Ben Auffarth | Leonid Kuligin

packt

Generative AI with LangChain

Second Edition

Build production-ready LLM applications and advanced agents using Python, LangChain, and LangGraph

Ben Auffarth

Leonid Kuligin

‹packt›

Generative AI with LangChain

Second Edition

Portfolio Director: Gebin George

Relationship Lead: Ali Abidi

Project Manager: Prajakta Naik

Content Engineer: Tanya D'cruz

Technical Editor: Irfa Ansari

Copy Editor: Safis Editing

Indexer: Manju Arasan

Proofreader: Tanya D'cruz

Production Designer: Ajay Patule

Growth Lead: Nimisha Dua

First published: December 2023

Second edition: May 2025

Production reference: 1190525

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83702-201-4

www.packtpub.com

To the mentors who guided me throughout my life—especially Tony Lindeberg, whose personal integrity and perseverance are a tremendous source of inspiration—and to my son, Nicholas, and my partner, Diane.

—Ben Auffarth

To my wife, Ksenia, whose unwavering love and optimism have been my constant support over all these years; to my mother-in-law, Tatyana, whose belief in me—even in my craziest endeavors—has been an incredible source of strength; and to my kids, Matvey and Milena: I hope you'll read it one day.

—Leonid Kuligin

Contributors

About the authors

Dr. Ben Auffarth, PhD, is an AI implementation expert with more than 15 years of work experience. As the founder of Chelsea AI Ventures, he specializes in helping small and medium enterprises implement enterprise-grade AI solutions that deliver tangible ROI. His systems have prevented millions in fraud losses and process transactions at sub-300ms latency. With a background in computational neuroscience, Ben brings rare depth to practical AI applications—from supercomputing brain models to production systems that combine technical excellence with business strategy.

First and foremost, I want to thank my co-author, Leo—a superstar coder—who's been patient throughout and always ready when advice was needed. This book also wouldn't be what it is without the people at Packt, especially Tanya, our editor, who offered sparks of insight and encouraging words whenever needed. Finally, the reviewers were very helpful and generous with their critiques, making sure we didn't miss anything. Any errors or oversights that remain are entirely mine.

Leonid Kuligin is a staff AI engineer at Google Cloud, working on generative AI and classical machine learning solutions, such as demand forecasting and optimization problems. Leonid is one of the key maintainers of Google Cloud integrations on LangChain and a visiting lecturer at CDTM (a joint institution of TUM and LMU). Prior to Google, Leonid gained more than 20 years of experience building B2C and B2B applications based on complex machine learning and data processing solutions—such as search, maps, and investment management—in German, Russian, and U.S. technology, financial, and retail companies.

I want to express my sincere gratitude to all my colleagues at Google with whom I had the pleasure and joy of working, and who supported me during the creation of this book and many other endeavors. Special thanks go to Max Tschochohei, Lucio Floretta, and Thomas Cliett. My appreciation also goes to the entire LangChain community, especially Harrison Chase, whose continuous development of the LangChain framework made my work as an engineer significantly easier.

About the reviewers

Max Tschochohei advises enterprise customers on how to realize their AI and ML ambitions on Google Cloud. As an engineering manager in Google Cloud Consulting, he leads teams of AI engineers on mission-critical customer projects. While his work spans the full range of AI products and solutions in the Google Cloud portfolio, he is particularly interested in agentic systems, machine learning operations, and healthcare applications of AI. Before joining Google in Munich, Max spent several years as a consultant, first with KPMG and later with the Boston Consulting Group. He also led the digital transformation of NTUC Enterprise, a Singapore government organization. Max holds a *PhD in Economics* from Coventry University.

Rany ElHousieny is an AI Solutions Architect and AI Engineering Manager with over two decades of experience in AI, NLP, and ML. Throughout his career, he has focused on the development and deployment of AI models, authoring multiple articles on AI systems architecture and ethical AI deployment. He has led groundbreaking projects at companies like Microsoft, where he spearheaded advancements in NLP and the Language Understanding Intelligent Service (LUIS). Currently, he plays a pivotal role at Clearwater Analytics, driving innovation in generative AI and AI-driven financial and investment management solutions.

Nicolas Bievre is a Machine Learning Engineer at Meta with extensive experience in AI, recommender systems, LLMs, and generative AI, applied to advertising and healthcare. He has held key AI leadership roles at Meta and PayPal, designing and implementing large-scale recommender systems used to personalize content for hundreds of millions of users. He graduated from Stanford University, where he published peer-reviewed research in leading AI and bioinformatics journals. Internationally recognized for his contributions, Nicolas has received awards such as the “Core Ads Growth Privacy” Award and the “Outre-Mer Outstanding Talent” Award. He also serves as an AI consultant to the French government and as a reviewer for top AI organizations.

Join our communities on Discord and Reddit

Have questions about the book or want to contribute to discussions on Generative AI and LLMs? Join our Discord server at `https://packt.link/4Bbd9` and our Reddit channel at `https://packt.link/wcYOQ` to connect, share, and collaborate with like-minded AI professionals.

Discord QR

Reddit QR

Table of Contents

Chapter 2: First Steps with LangChain 25

Preface

With **Large Language Models (LLMs)** now powering everything from customer service chatbots to sophisticated code generation systems, generative AI has rapidly transformed from a research lab curiosity to a production workhorse. Yet a significant gap exists between experimental prototypes and production-ready AI applications. According to industry research, while enthusiasm for generative AI is high, over 30% of projects fail to move beyond proof of concept due to reliability issues, evaluation complexity, and integration challenges. The LangChain framework has emerged as an essential bridge across this divide, providing developers with the tools to build robust, scalable, and practical LLM applications.

This book is designed to help you close that gap. It's your practical guide to building LLM applications that actually work in production environments. We focus on real-world problems that derail most generative AI projects: inconsistent outputs, difficult debugging, fragile tool integrations, and scaling bottlenecks. Through hands-on examples and tested patterns using LangChain, LangGraph, and other tools in the growing generative AI ecosystem, you'll learn to build systems that your organization can confidently deploy and maintain to solve real problems.

Who this book is for

This book is primarily written for software developers with basic Python knowledge who want to build production-ready applications using LLMs. You don't need extensive machine learning expertise, but some familiarity with AI concepts will help you move more quickly through the material. By the end of the book, you'll be confidently implementing advanced LLM architectures that would otherwise require specialized AI knowledge.

If you're a data scientist transitioning into LLM application development, you'll find the practical implementation patterns especially valuable, as they bridge the gap between experimental notebooks and deployable systems. The book's structured approach to RAG implementation, evaluation frameworks, and observability practices addresses the common frustrations you've likely encountered when trying to scale promising prototypes into reliable services.

For technical decision-makers evaluating LLM technologies within their organizations, this book offers strategic insight into successful LLM project implementations. You'll understand the architectural patterns that differentiate experimental systems from production-ready ones, learn to identify high-value use cases, and discover how to avoid the integration and scaling issues that cause most projects to fail. The book provides clear criteria for evaluating implementation approaches and making informed technology decisions.

What this book covers

Chapter 1, *The Rise of Generative AI, From Language Models to Agents*, introduces the modern LLM landscape and positions LangChain as the framework for building production-ready AI applications. You'll learn about the practical limitations of basic LLMs and how frameworks like LangChain help with standardization and overcoming these challenges. This foundation will help you make informed decisions about which agent technologies to implement for your specific use cases.

Chapter 2, *First Steps with LangChain*, gets you building immediately with practical, hands-on examples. You'll set up a proper development environment, understand LangChain's core components (model interfaces, prompts, templates, and LCEL), and create simple chains. The chapter shows you how to run both cloud-based and local models, giving you options to balance cost, privacy, and performance based on your project needs. You'll also explore simple multimodal applications that combine text with visual understanding. These fundamentals provide the building blocks for increasingly sophisticated AI applications.

Chapter 3, *Building Workflows with LangGraph*, dives into creating complex workflows with LangChain and LangGraph. You'll learn to build workflows with nodes and edges, including conditional edges for branching based on state. The chapter covers output parsing, error handling, prompt engineering techniques (zero-shot and dynamic few-shot prompting), and working with long contexts using Map-Reduce patterns. You'll also implement memory mechanisms for managing chat history. These skills address why many LLM applications fail in real-world conditions and give you the tools to build systems that perform reliably.

Chapter 4, *Building Intelligent RAG Systems*, addresses the "hallucination problem" by grounding LLMs in reliable external knowledge. You'll master vector stores, document processing, and retrieval strategies that improve response accuracy. The chapter's corporate documentation chatbot project demonstrates how to implement enterprise-grade RAG pipelines that maintain consistency and compliance—a capability that directly addresses data quality concerns cited in industry surveys. The troubleshooting section covers seven common RAG failure points and provides practical solutions for each.

Chapter 5, *Building Intelligent Agents*, tackles tool use fragility—identified as a core bottleneck in agent autonomy. You'll implement the ReACT pattern to improve agent reasoning and decision-making, develop robust custom tools, and build error-resilient tool calling processes. Through practical examples like generating structured outputs and building a research agent, you'll understand what agents are and implement your first plan-and-solve agent with LangGraph, setting the stage for more advanced agent architectures.

Chapter 6, *Advanced Applications and Multi-Agent Systems*, covers architectural patterns for agentic AI applications. You'll explore multi-agent architectures and ways to organize communication between agents, implementing an advanced agent with self-reflection that uses tools to answer complex questions. The chapter also covers LangGraph streaming, advanced control flows, adaptive systems with humans in the loop, and the Tree-of-Thoughts pattern. You'll learn about memory mechanisms in LangChain and LangGraph, including caches and stores, equipping you to create systems capable of tackling problems too complex for single-agent approaches—a key capability of production-ready systems.

Chapter 7, *Software Development and Data Analysis Agents*, demonstrates how natural language has become a powerful interface for programming and data analysis. You'll implement LLM-based solutions for code generation, code retrieval with RAG, and documentation search. These examples show how to integrate LLM agents into existing development and data workflows, illustrating how they complement rather than replace traditional programming skills.

Chapter 8, *Evaluation and Testing*, outlines methodologies for assessing LLM applications before production deployment. You'll learn about system-level evaluation, evaluation-driven design, and both offline and online methods. The chapter provides practical examples for implementing correctness evaluation using exact matches and LLM-as-a-judge approaches and demonstrates tools like LangSmith for comprehensive testing and monitoring. These techniques directly increase reliability and help justify the business value of your LLM applications.

Chapter 9, *Observability and Production Deployment*, provides guidelines for deploying LLM applications into production, focusing on system design, scaling strategies, monitoring, and ensuring high availability. The chapter covers logging, API design, cost optimization, and redundancy strategies specific to LLMs. You'll explore the Model Context Protocol (MCP) and learn how to implement observability practices that address the unique challenges of deploying generative AI systems. The practical deployment patterns in this chapter help you avoid common pitfalls that prevent many LLM projects from reaching production.

Chapter 10, *The Future of LLM Applications*, looks ahead to emerging trends, evolving architectures, and ethical considerations in generative AI. The chapter explores new technologies, market developments, potential societal impacts, and guidelines for responsible development. You'll gain insight into how the field is likely to evolve and how to position your skills and applications for future advancements, completing your journey from basic LLM understanding to building and deploying production-ready, future-proof AI systems.

To get the most out of this book

Before diving in, it's helpful to ensure you have a few things in place to make the most of your learning experience. This book is designed to be hands-on and practical, so having the right environment, tools, and mindset will help you follow along smoothly and get the full value from each chapter. Here's what we recommend:

- **Environment requirements**: Set up a development environment with Python 3.10+ on any major operating system (Windows, macOS, or Linux). All code examples are cross-platform compatible and thoroughly tested.
- **API access (optional but recommended)**: While we demonstrate using open-source models that can run locally, having access to commercial API providers like OpenAI, Anthropic, or other LLM providers will allow you to work with more powerful models. Many examples include both local and API-based approaches, so you can choose based on your budget and performance needs.
- **Learning approach**: We recommend typing the code yourself rather than copying and pasting. This hands-on practice reinforces learning and encourages experimentation. Each chapter builds on concepts introduced earlier, so working through them sequentially will give you the strongest foundation.
- **Background knowledge**: Basic Python proficiency is required, but no prior experience with machine learning or LLMs is necessary. We explain key concepts as they arise. If you're already familiar with LLMs, you can focus on the implementation patterns and production-readiness aspects that distinguish this book.

Software/Hardware covered in the book
Python 3.10+
LangChain 0.3.1+
LangGraph 0.2.10+
Various LLM providers (Anthropic, Google, OpenAI, local models)

You'll find detailed guidance on environment setup in *Chapter 1*, along with clear explanations and step-by-step instructions to help you get started. We strongly recommend following these setup steps as outlined—given the fast-moving nature of LangChain, LangGraph and the broader ecosystem, skipping them might lead to avoidable issues down the line.

Download the example code files

The code bundle for the book is hosted on GitHub at `https://github.com/benman1/generative_ai_with_langchain`. We recommend typing the code yourself or using the repository as you progress through the chapters. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing`. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: `https://packt.link/gbp/9781837022014`.

Conventions used

There are a number of text conventions used throughout this book.

`CodeInText`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Let's also restore from the initial checkpoint for `thread-a`. We'll see that we start with an empty history:"

A block of code is set as follows:

```
checkpoint_id = checkpoints[-1].config["configurable"]["checkpoint_id"]
_ = graph.invoke(
   [HumanMessage(content="test")],
   config={"configurable": {"thread_id": "thread-a", "checkpoint_id":
checkpoint_id}})
```

Any command-line input or output is written as follows:

```
$ pip install langchain langchain-openai
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: " The Google Research team introduced the **Chain-of-Thought (CoT)** technique early in 2022."

Warnings or important notes appear like this.

Tips and tricks appear like this.

Get in touch

Subscribe to AI_Distilled, the go-to newsletter for AI professionals, researchers, and innovators, at `https://packt.link/Q5UyU`.

Feedback from our readers is always welcome.

If you find any errors or have suggestions, please report them preferably through issues on GitHub, the discord chat, or the errata submission form on the Packt website.

For issues on GitHub, see `https://github.com/benman1/generative_ai_with_langchain/issues`.

If you have questions about the book's content, or bespoke projects, feel free to contact us at `ben@chelseaai.co.uk`.

General feedback: Email `feedback@packtpub.com` and mention the book's title in the subject of your message. If you have questions about any aspect of this book, please email us at `questions@packtpub.com`.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit `http://www.packtpub.com/submit-errata`, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packtpub.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `http://authors.packtpub.com/`.

Share your thoughts

Once you've read *Generative AI with LangChain, Second Edition,* we'd love to hear your thoughts! Please `click here to go straight to the Amazon review page` for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily.

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below:

`https://packt.link/free-ebook/9781837022014`

2. Submit your proof of purchase.
3. That's it! We'll send your free PDF and other benefits to your email directly.

1

The Rise of Generative AI: From Language Models to Agents

The gap between experimental and production-ready agents is stark. According to LangChain's State of Agents report, performance quality is the #1 concern among 51% of companies using agents, yet only 39.8% have implemented proper evaluation systems. Our book bridges this gap on two fronts: first, by demonstrating how LangChain and LangSmith provide robust testing and observability solutions; second, by showing how LangGraph's state management enables complex, reliable multi-agent systems. You'll find production-tested code patterns that leverage each tool's strengths for enterprise-scale implementation and extend basic RAG into robust knowledge systems.

LangChain accelerates time-to-market with readily available building blocks, unified vendor APIs, and detailed tutorials. Furthermore, LangChain and LangSmith debugging and tracing functionalities simplify the analysis of complex agent behavior. Finally, LangGraph has excelled in executing its philosophy behind agentic AI – it allows a developer to give a **large language model (LLM)** partial control flow over the workflow (and to manage the level of how much control an LLM should have), while still making agentic workflows reliable and well-performant.

In this chapter, we'll explore how LLMs have evolved into the foundation for agentic AI systems and how frameworks like LangChain and LangGraph transform these models into production-ready applications. We'll also examine the modern LLM landscape, understand the limitations of raw LLMs, and introduce the core concepts of agentic applications that form the basis for the hands-on development we'll tackle throughout this book.

In a nutshell, the following topics will be covered in this book:

- The modern LLM landscape
- From models to agentic applications
- Introducing LangChain

The modern LLM landscape

Artificial intelligence (AI) has long been a subject of fascination and research, but recent advancements in generative AI have propelled it into mainstream adoption. Unlike traditional AI systems that classify data or make predictions, generative AI can create new content—text, images, code, and more—by leveraging vast amounts of training data.

The generative AI revolution was catalyzed by the 2017 introduction of the transformer architecture, which enabled models to process text with unprecedented understanding of context and relationships. As researchers scaled these models from millions to billions of parameters, they discovered something remarkable: larger models didn't just perform incrementally better—they exhibited entirely new emergent capabilities like few-shot learning, complex reasoning, and creative generation that weren't explicitly programmed. Eventually, the release of ChatGPT in 2022 marked a turning point, demonstrating these capabilities to the public and sparking widespread adoption.

The landscape shifted again with the open-source revolution led by models like Llama and Mistral, democratizing access to powerful AI beyond the major tech companies. However, these advanced capabilities came with significant limitations—models couldn't reliably use tools, reason through complex problems, or maintain context across interactions. This gap between raw model power and practical utility created the need for specialized frameworks like LangChain that transform these models from impressive text generators into functional, production-ready agents capable of solving real-world problems.

Key terminologies

Tools: External utilities or functions that AI models can use to interact with the world. Tools allow agents to perform actions like searching the web, calculating values, or accessing databases to overcome LLMs' inherent limitations.

Memory: Systems that allow AI applications to store and retrieve information across interactions. Memory enables contextual awareness in conversations and complex workflows by tracking previous inputs, outputs, and important information.

Reinforcement learning from human feedback (RLHF): A training technique where AI models learn from direct human feedback, optimizing their performance to align with human preferences. RLHF helps create models that are more helpful, safe, and aligned with human values.

Agents: AI systems that can perceive their environment, make decisions, and take actions to accomplish goals. In LangChain, agents use LLMs to interpret tasks, choose appropriate tools, and execute multi-step processes with minimal human intervention.

Year	Development	Key Features
1990s	IBM Alignment Models	Statistical machine translation
2000s	Web-scale datasets	Large-scale statistical models
2009	Statistical models dominate	Large-scale text ingestion
2012	Deep learning gains traction	Neural networks outperform statistical models
2016	Neural Machine Translation (NMT)	Seq2seq deep LSTMs replace statistical methods
2017	Transformer architecture	Self-attention revolutionizes NLP
2018	BERT and GPT-1	Transformer-based language understanding and generation
2019	GPT-2	Large-scale text generation, public awareness increases
2020	GPT-3	API-based access, state-of-the-art performance
2022	ChatGPT	Mainstream adoption of LLMs
2023	Large Multimodal Models (LMMs)	AI models process text, images, and audio

2024	OpenAI o1	Stronger reasoning capabilities
2025	DeepSeek R1	Open-weight, large-scale AI model

Table 1.1: A timeline of major developments in language models

The field of LLMs is rapidly evolving, with multiple models competing in terms of performance, capabilities, and accessibility. Each provider brings distinct advantages, from OpenAI's advanced general-purpose AI to Mistral's open-weight, high-efficiency models. Understanding the differences between these models helps practitioners make informed decisions when integrating LLMs into their applications.

Model comparison

The following points outline key factors to consider when comparing different LLMs, focusing on their accessibility, size, capabilities, and specialization:

- **Open-source vs. closed-source models**: Open-source models like Mistral and LLaMA provide transparency and the ability to run locally, while closed-source models like GPT-4 and Claude are accessible through APIs. Open-source LLMs can be downloaded and modified, enabling developers and researchers to investigate and build upon their architectures, though specific usage terms may apply.
- **Size and capabilities**: Larger models generally offer better performance but require more computational resources. This makes smaller models great for use on devices with limited computing power or memory, and can be significantly cheaper to use. **Small language models (SLMs)** have a relatively small number of parameters, typically using millions to a few billion parameters, as opposed to LLMs, which can have hundreds of billions or even trillions of parameters.
- **Specialized models**: Some LLMs are optimized for specific tasks, such as code generation (for example, Codex) or mathematical reasoning (e.g., Minerva).

The increase in the scale of language models has been a major driving force behind their impressive performance gains. However, recently there has been a shift in architecture and training methods that has led to better parameter efficiency in terms of performance.

Model scaling laws

Empirically derived scaling laws predict the performance of LLMs based on the given training budget, dataset size, and the number of parameters. If true, this means that highly powerful systems will be concentrated in the hands of Big Tech, however, we have seen a significant shift over recent months.

The **KM scaling law**, proposed by Kaplan et al., derived through empirical analysis and fitting of model performance with varied data sizes, model sizes, and training compute, presents power-law relationships, indicating a strong codependence between model performance and factors such as model size, dataset size, and training compute.

The **Chinchilla scaling law**, proposed by the Google DeepMind team, involved experiments with a wider range of model sizes and data sizes. It suggests an optimal allocation of compute budget to model size and data size, which can be determined by optimizing a specific loss function under a constraint.

However, future progress may depend more on model architecture, data cleansing, and model algorithmic innovation rather than sheer size. For example, models such as phi, first presented in *Textbooks Are All You Need* (2023, Gunasekar et al.), with about 1 billion parameters, showed that models can – despite a smaller scale – achieve high accuracy on evaluation benchmarks. The authors suggest that improving data quality can dramatically change the shape of scaling laws.

Further, there is a body of work on simplified model architectures, which have substantially fewer parameters and only modestly drop accuracy (for example, *One Wide Feedforward is All You Need*, Pessoa Pires et al., 2023). Additionally, techniques such as fine-tuning, quantization, distillation, and prompting techniques can enable smaller models to leverage the capabilities of large foundations without replicating their costs. To compensate for model limitations, tools like search engines and calculators have been incorporated into agents, and multi-step reasoning strategies, plugins, and extensions may be increasingly used to expand capabilities.

The future could see the co-existence of massive, general models with smaller and more accessible models that provide faster and cheaper training, maintenance, and inference.

Let's now discuss a comparative overview of various LLMs, highlighting their key characteristics and differentiating factors. We'll delve into aspects such as open-source vs. closed-source models, model size and capabilities, and specialized models. By understanding these distinctions, you can select the most suitable LLM for your specific needs and applications.

LLM provider landscape

You can access LLMs from major providers like OpenAI, Google, and Anthropic, along with a growing number of others, through their websites or APIs. As the demand for LLMs grows, numerous providers have entered the space, each offering models with unique capabilities and trade-offs. Developers need to understand the various access options available for integrating these powerful models into their applications. The choice of provider will significantly impact development experience, performance characteristics, and operational costs.

The table below provides a comparative overview of leading LLM providers and examples of the models they offer:

Provider	Notable models	Key features and strengths
OpenAI	GPT-4o, GPT-4.5; o1; o3-mini	Strong general performance, proprietary models, advanced reasoning; multimodal reasoning across text, audio, vision, and video in real time
Anthropic	Claude 3.7 Sonnet; Claude 3.5 Haiku	Toggle between real-time responses and extended "thinking" phases; outperforms OpenAI's o1 in coding benchmarks
Google	Gemini 2.5, 2.0 (flash and pro), Gemini 1.5	Low latency and costs, large context window (up to 2M tokens), multimodal inputs and outputs, reasoning capabilities
Cohere	Command R, Command R Plus	Retrieval-augmented generation, enterprise AI solutions
Mistral AI	Mistral Large; Mistral 7B	Open weights, efficient inference, multilingual support
AWS	Titan	Enterprise-scale AI models, optimized for the AWS cloud

DeepSeek	R1	Maths-first: solves Olympiad-level problems; cost-effective, optimized for multilingual and programming tasks
Together AI	Infrastructure for running open models	Competitive pricing; growing marketplace of models

Table 1.2: Comparative overview of major LLM providers and their flagship models for LangChain implementation

Other organizations develop LLMs but do not necessarily provide them through **application programming interfaces** (**APIs**) to developers. For example, Meta AI develops the very influential Llama model series, which has strong reasoning, code-generation capabilities, and is released under an open-source license.

There is a whole zoo of open-source models that you can access through Hugging Face or through other providers. You can even download these open-source models, fine-tune them, or fully train them. We'll try this out practically starting in *Chapter 2*.

Once you've selected an appropriate model, the next crucial step is understanding how to control its behavior to suit your specific application needs. While accessing a model gives you computational capability, it's the choice of generation parameters that transforms raw model power into tailored output for different use cases within your applications.

Now that we've covered the LLM provider landscape, let's discuss another critical aspect of LLM implementation: licensing considerations. The licensing terms of different models significantly impact how you can use them in your applications.

Licensing

LLMs are available under different licensing models that impact how they can be used in practice. Open-source models like Mixtral and BERT can be freely used, modified, and integrated into applications. These models allow developers to run them locally, investigate their behavior, and build upon them for both research and commercial purposes.

In contrast, proprietary models like GPT-4 and Claude are accessible only through APIs, with their internal workings kept private. While this ensures consistent performance and regular updates, it means depending on external services and typically incurring usage costs.

Some models like Llama 2 take a middle ground, offering permissive licenses for both research and commercial use while maintaining certain usage conditions. For detailed information about specific model licenses and their implications, refer to the documentation of each model or consult the model openness framework: `https://isitopen.ai/`.

The **model openness framework** (**MOF**) evaluates language models based on criteria such as access to model architecture details, training methodology and hyperparameters, data sourcing and processing information, documentation around development decisions, ability to evaluate model workings, biases, and limitations, code modularity, published model card, availability of servable model, option to run locally, source code availability, and redistribution rights.

In general, open-source licenses promote wide adoption, collaboration, and innovation around the models, benefiting both research and commercial development. Proprietary licenses typically give companies exclusive control but may limit academic research progress. Non-commercial licenses often restrict commercial use while enabling research.

By making knowledge and knowledge work more accessible and adaptable, generative AI models have the potential to level the playing field and create new opportunities for people from all walks of life.

The evolution of AI has brought us to a pivotal moment where AI systems can not only process information but also take autonomous action. The next section explores the transformation from basic language models to more complex, and finally, fully agentic applications.

The information provided about AI model licensing is for educational purposes only and does not constitute legal advice. Licensing terms vary significantly and evolve rapidly. Organizations should consult qualified legal counsel regarding specific licensing decisions for their AI implementations.

From models to agentic applications

As discussed so far, LLMs have been demonstrating remarkable fluency in natural language processing. However, as impressive as they are, they remain fundamentally *reactive* rather than *proactive*. They lack the ability to take independent actions, interact meaningfully with external systems, or autonomously achieve complex objectives.

To unlock the next phase of AI capabilities, we need to move beyond passive text generation and toward **agentic AI**—systems that can plan, reason, and take action to accomplish tasks with minimal human intervention. Before exploring the potential of agentic AI, it's important to first understand the core limitations of LLMs that necessitate this evolution.

Limitations of traditional LLMs

Despite their advanced language capabilities, LLMs have inherent constraints that limit their effectiveness in real-world applications:

1. **Lack of true understanding**: LLMs generate human-like text by predicting the next most likely word based on statistical patterns in training data. However, they do not understand meaning in the way humans do. This leads to hallucinations—confidently stating false information as fact—and generating plausible but incorrect, misleading, or nonsensical outputs. As Bender et al. (2021) describe, LLMs function as "stochastic parrots"—repeating patterns without genuine comprehension.
2. **Struggles with complex reasoning and problem-solving**: While LLMs excel at retrieving and reformatting knowledge, they struggle with multi-step reasoning, logical puzzles, and mathematical problem-solving. They often fail to break down problems into sub-tasks or synthesize information across different contexts. Without explicit prompting techniques like chain-of-thought reasoning, their ability to deduce or infer remains unreliable.
3. **Outdated knowledge and limited external access**: LLMs are trained on static datasets and do not have real-time access to current events, dynamic databases, or live information sources. This makes them unsuitable for tasks requiring up-to-date knowledge, such as financial analysis, breaking news summaries, or scientific research requiring the latest findings.
4. **No native tool use or action-taking abilities**: LLMs operate in isolation—they cannot interact with APIs, retrieve live data, execute code, or modify external systems. This lack of tool integration makes them less effective in scenarios that require real-world actions, such as conducting web searches, automating workflows, or controlling software systems.
5. **Bias, ethical concerns, and reliability issues:** Because LLMs learn from large datasets that may contain biases, they can unintentionally reinforce ideological, social, or cultural biases. Importantly, even with open-source models, accessing and auditing the complete training data to identify and mitigate these biases remains challenging for most practitioners. Additionally, they can generate misleading or harmful information without understanding the ethical implications of their outputs.

6. **Computational costs and efficiency challenges**: Deploying and running LLMs at scale requires **significant** computational resources, making them costly and energy-intensive. Larger models can also introduce latency, slowing response times in real-time applications.

To overcome these limitations, AI systems must evolve from passive text generators into active agents that can plan, reason, and interact with their environment. This is where agentic AI comes in—integrating LLMs with tool use, decision-making mechanisms, and autonomous execution capabilities to enhance their functionality.

While frameworks like LangChain provide comprehensive solutions to LLM limitations, understanding fundamental prompt engineering techniques remains valuable. Approaches like few-shot learning, chain-of-thought, and structured prompting can significantly enhance model performance for specific tasks. *Chapter 3* will cover these techniques in detail, showing how LangChain helps standardize and optimize prompting patterns while minimizing the need for custom prompt engineering in every application.

The next section explores how agentic AI extends the capabilities of traditional LLMs and unlocks new possibilities for automation, problem-solving, and intelligent decision-making.

Understanding LLM applications

LLM applications represent the bridge between raw model capability and practical business value. While LLMs possess impressive language processing abilities, they require thoughtful integration to deliver real-world solutions. These applications broadly fall into two categories: complex integrated applications and autonomous agents.

Complex integrated applications enhance human workflows by integrating LLMs into existing processes, including:

- Decision support systems that provide analysis and recommendations
- Content generation pipelines with human review
- Interactive tools that augment human capabilities
- Workflow automation with human oversight

Autonomous agents operate with minimal human intervention, further augmenting workflows through LLM integration. Examples include:

- Task automation agents that execute defined workflows
- Information gathering and analysis systems
- Multi-agent systems for complex task coordination

LangChain provides frameworks for both integrated applications and autonomous agents, offering flexible components that support various architectural choices. This book will explore both approaches, demonstrating how to build reliable, production-ready systems that match your specific requirements.

Autonomous systems of agents are potentially very powerful, and it's therefore worthwhile exploring them a bit more.

Understanding AI agents

It is sometimes joked that AI is just a fancy word for ML, or AI is ML in a suit, as illustrated in this image; however, there's more to it, as we'll see.

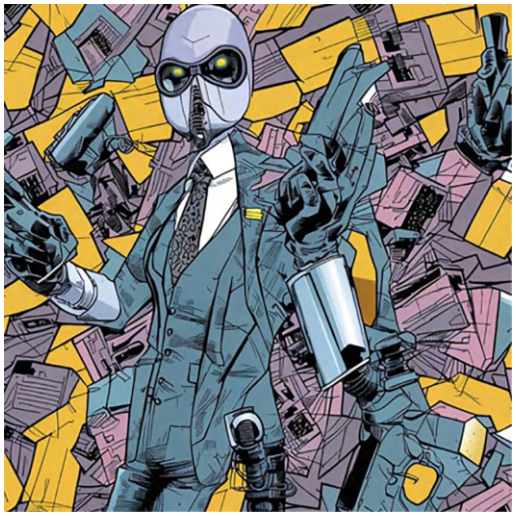

Figure 1.1: ML in a suit. Generated by a model on replicate.com, Diffusers Stable Diffusion v2.1

An AI agent represents the bridge between raw cognitive capability and practical action. While an LLM possesses vast knowledge and processing ability, it remains fundamentally reactive without agency. AI agents transform this passive capability into active utility through structured workflows that parse requirements, analyze options, and execute actions.

Agentic AI enables autonomous systems to make decisions and act independently, with minimal human intervention. Unlike deterministic systems that follow fixed rules, agentic AI relies on patterns and likelihoods to make informed choices. It functions through a network of autonomous software components called agents, which learn from user behavior and large datasets to improve over time.

Agency in AI refers to a system's ability to act independently to achieve goals. True agency means an AI system can perceive its environment, make decisions, act, and adapt over time by learning from interactions and feedback. The distinction between raw AI and agents parallels the difference between knowledge and expertise. Consider a brilliant researcher who understands complex theories but struggles with practical application. An agent system adds the crucial element of purposeful action, turning abstract capability into concrete results.

In the context of LLMs, agentic AI involves developing systems that act autonomously, understand context, adapt to new information, and collaborate with humans to solve complex challenges. These AI agents leverage LLMs to process information, generate responses, and execute tasks based on defined objectives.

Particularly, AI agents extend the capabilities of LLMs by integrating memory, tool use, and decision-making frameworks. These agents can:

- Retain and recall information across interactions.
- Utilize external tools, APIs, and databases.
- Plan and execute multi-step workflows.

The value of agency lies in reducing the need for constant human oversight. Instead of manually prompting an LLM for every request, an agent can proactively execute tasks, react to new data, and integrate with real-world applications.

AI agents are systems designed to act on behalf of users, leveraging LLMs alongside external tools, memory, and decision-making frameworks. The hope behind AI agents is that they can automate complex workflows, reducing human effort while increasing efficiency and accuracy. By allowing systems to act autonomously, agents promise to unlock new levels of automation in AI-driven applications. But are the hopes justified?

Despite their potential, AI agents face significant challenges:

- **Reliability**: Ensuring agents make correct, context-aware decisions without supervision is difficult.
- **Generalization**: Many agents work well in narrow domains but struggle with open-ended, multi-domain tasks.
- **Lack of trust**: Users must trust that agents will act responsibly, avoid unintended actions, and respect privacy constraints.
- **Coordination complexity**: Multi-agent systems often suffer from inefficiencies and miscommunication when executing tasks collaboratively.

Production-ready agent systems must address not just theoretical challenges but practical implementation hurdles like:

- Rate limitations and API quotas
- Token context overflow errors
- Hallucination management
- Cost optimization

LangChain and LangSmith provide robust solutions for these challenges, which we'll explore in depth in *Chapter 8* and *Chapter 9*. These chapters will cover how to build reliable, observable AI systems that can operate at an enterprise scale.

When developing agent-based systems, therefore, several key factors require careful consideration:

- **Value generation**: Agents must provide a clear utility that outweighs their costs in terms of setup, maintenance, and necessary human oversight. This often means starting with well-defined, high-value tasks where automation can demonstrably improve outcomes.
- **Trust and safety**: As agents take on more responsibility, establishing and maintaining user trust becomes crucial. This encompasses both technical reliability and transparent operation that allows users to understand and predict agent behavior.
- **Standardization**: As the agent ecosystem grows, standardized interfaces and protocols become essential for interoperability. This parallels the development of web standards that enabled the growth of internet applications.

While early AI systems focused on pattern matching and predefined templates, modern AI agents demonstrate emergent capabilities such as reasoning, problem-solving, and long-term planning. Today's AI agents integrate LLMs with interactive environments, enabling them to function autonomously in complex domains.

The development of agent-based AI is a natural progression from statistical models to deep learning and now to reasoning-based systems. Modern AI agents leverage multimodal capabilities, reinforcement learning, and memory-augmented architectures to adapt to diverse tasks. This evolution marks a shift from predictive models to truly autonomous systems capable of dynamic decision-making.

Looking ahead, AI agents will continue to refine their ability to reason, plan, and act within structured and unstructured environments. The rise of open-weight models, combined with advances in agent-based AI, will likely drive the next wave of innovations in AI, expanding its applications across science, engineering, and everyday life.

With frameworks like LangChain, developers can build complex and agentic structured systems that overcome the limitations of raw LLMs. It offers built-in solutions for memory management, tool integration, and multi-step reasoning that align with the ecosystem model presented here. In the next section we will explore how LangChain facilitates the development of production-ready AI agents.

Introducing LangChain

LangChain exists as both an open-source framework and a venture-backed company. The framework, introduced in 2022 by Harrison Chase, streamlines the development of LLM-powered applications with support for multiple programming languages including Python, JavaScript/TypeScript, Go, Rust, and Ruby.

The company behind the framework, LangChain, Inc., is based in San Francisco and has secured significant venture funding through multiple rounds, including a Series A in February 2024. With 11-50 employees, the company maintains and expands the framework while offering enterprise solutions for LLM application development.

While the core framework remains open source, the company provides additional enterprise features and support for commercial users. Both share the same mission: accelerating LLM application development by providing robust tools and infrastructure.

Modern LLMs are undeniably powerful, but their practical utility in production applications is constrained by several inherent limitations. Understanding these challenges is essential for appreciating why frameworks like LangChain have become indispensable tools for AI developers.

Challenges with raw LLMs

Despite their impressive capabilities, LLMs face fundamental constraints that create significant hurdles for developers building real-world applications:

1. **Context window limitations**: LLMs process text as tokens (subword units), not complete words. For example, "LangChain" might be processed as two tokens: "Lang" and "Chain." Every LLM has a fixed context window—the maximum number of tokens it can process at once—typically ranging from 2,000 to 128,000 tokens. This creates several practical challenges:

    a. **Document processing**: Long documents must be chunked effectively to fit within context limits

   b. **Conversation history**: Maintaining information across extended conversations requires careful memory management
   c. **Cost management**: Most providers charge based on token count, making efficient token use a business imperative

   These constraints directly impact application architecture, making techniques like RAG (which we'll explore in *Chapter 4*) essential for production systems.

2. **Limited tool orchestration**: While many modern LLMs offer native tool-calling capabilities, they lack the infrastructure to discover appropriate tools, execute complex workflows, and manage tool interactions across multiple turns. Without this orchestration layer, developers must build custom solutions for each integration.
3. **Task coordination challenges**: Managing multi-step workflows with LLMs requires structured control mechanisms. Without them, complex processes involving sequential reasoning or decision-making become difficult to implement reliably.

Tools in this context refer to functional capabilities that extend an LLM's reach: web browsers for searching the internet, calculators for precise mathematics, coding environments for executing programs, or APIs for accessing external services and databases. Without these tools, LLMs remain confined to operating within their training knowledge, unable to perform real-world actions or access current information.

These fundamental limitations create three key challenges for developers working with raw LLM APIs, as demonstrated in the following table.

Challenge	Description	Impact
Reliability	Detecting hallucinations and validating outputs	Inconsistent results that may require human verification
Resource Management	Handling context windows and rate limits	Implementation complexity and potential cost overruns
Integration Complexity	Building connections to external tools and data sources	Extended development time and maintenance burden

Table 1.3: Three key developer challenges

LangChain addresses these challenges by providing a structured framework with tested solutions, simplifying AI application development and enabling more sophisticated use cases.

How LangChain enables agent development

LangChain provides the foundational infrastructure for building sophisticated AI applications through its modular architecture and composable patterns. With the evolution to version 0.3, LangChain has refined its approach to creating intelligent systems:

- **Composable workflows**: The **LangChain Expression Language (LCEL)** allows developers to break down complex tasks into modular components that can be assembled and reconfigured. This composability enables systematic reasoning through the orchestration of multiple processing steps.
- **Integration ecosystem**: LangChain offers battle-tested abstract interfaces for all generative AI components (LLMs, embeddings, vector databases, document loaders, search engines). This lets you build applications that can easily switch between providers without rewriting core logic.
- **Unified model access**: The framework provides consistent interfaces to diverse language and embedding models, allowing seamless switching between providers while maintaining application logic.

While earlier versions of LangChain handled memory management directly, version 0.3 takes a more specialized approach to application development:

- **Memory and state management**: For applications requiring persistent context across interactions, LangGraph now serves as the recommended solution. LangGraph maintains conversation history and application state with purpose-built persistence mechanisms.
- **Agent architecture**: Though LangChain contains agent implementations, LangGraph has become the preferred framework for building sophisticated agents. It provides:
    - Graph-based workflow definition for complex decision paths
    - Persistent state management across multiple interactions
    - Streaming support for real-time feedback during processing
    - Human-in-the-loop capabilities for validation and corrections

Together, LangChain and its companion projects like LangGraph and LangSmith form a comprehensive ecosystem that transforms LLMs from simple text generators into systems capable of sophisticated real-world tasks, combining strong abstractions with practical implementation patterns optimized for production use.

Exploring the LangChain architecture

LangChain's philosophy centers on composability and modularity. Rather than treating LLMs as standalone services, LangChain views them as components that can be combined with other tools and services to create more capable systems. This approach is built on several principles:

- **Modular architecture**: Every component is designed to be reusable and interchangeable, allowing developers to integrate LLMs seamlessly into various applications. This modularity extends beyond LLMs to include numerous building blocks for developing complex generative AI applications.
- **Support for agentic workflows**: LangChain offers best-in-class APIs that allow you to develop sophisticated agents quickly. These agents can make decisions, use tools, and solve problems with minimal development overhead.
- **Production readiness**: The framework provides built-in capabilities for tracing, evaluation, and deployment of generative AI applications, including robust building blocks for managing memory and persistence across interactions.
- **Broad vendor ecosystem**: LangChain offers battle-tested abstract interfaces for all generative AI components (LLMs, embeddings, vector databases, document loaders, search engines, etc.). Vendors develop their own integrations that comply with these interfaces, allowing you to build applications on top of any third-party provider and easily switch between them.

It's worth noting that there've been major changes since LangChain version 0.1 when the first edition of this book was written. While early versions attempted to handle everything, LangChain version 0.3 focuses on excelling at specific functions with companion projects handling specialized needs. LangChain manages model integration and workflows, while LangGraph handles stateful agents and LangSmith provides observability.

LangChain's memory management, too, has gone through major changes. Memory mechanisms within the base LangChain library have been deprecated in favor of LangGraph for persistence, and while agents are present, LangGraph is the recommended approach for their creation in version 0.3. However, models and tools continue to be fundamental to LangChain's functionality. In *Chapter 3*, we'll explore LangChain and LangGraph's memory mechanisms.

To translate model design principles into practical tools, LangChain has developed a comprehensive ecosystem of libraries, services, and applications. This ecosystem provides developers with everything they need to build, deploy, and maintain sophisticated AI applications. Let's examine the components that make up this thriving environment and how they've gained adoption across the industry.

Ecosystem

LangChain has achieved impressive ecosystem metrics, demonstrating strong market adoption with over 20 million monthly downloads and powering more than 100,000 applications. Its open-source community is thriving, evidenced by 100,000+ GitHub stars and contributions from over 4,000 developers. This scale of adoption positions LangChain as a leading framework in the AI application development space, particularly for building reasoning-focused LLM applications. The framework's modular architecture (with components like LangGraph for agent workflows and LangSmith for monitoring) has clearly resonated with developers building production AI systems across various industries.

Core libraries

- LangChain (Python): Reusable components for building LLM applications
- LangChain.js: JavaScript/TypeScript implementation of the framework
- LangGraph (Python): Tools for building LLM agents as orchestrated graphs
- LangGraph.js: JavaScript implementation for agent workflows

Platform services

- LangSmith: Platform for debugging, testing, evaluating, and monitoring LLM applications
- LangGraph: Infrastructure for deploying and scaling LangGraph agents

Applications and extensions

- ChatLangChain: Documentation assistant for answering questions about the framework
- Open Canvas: Document and chat-based UX for writing code/markdown (TypeScript)
- OpenGPTs: Open source implementation of OpenAI's GPTs API
- Email assistant: AI tool for email management (Python)
- Social media agent: Agent for content curation and scheduling (TypeScript)

The ecosystem provides a complete solution for building reasoning-focused AI applications: from core building blocks to deployment platforms to reference implementations. This architecture allows developers to use components independently or stack them for fuller and more complete solutions.

From customer testimonials and company partnerships, LangChain is being adopted by enterprises like Rakuten, Elastic, Ally, and Adyen. Organizations report using LangChain and LangSmith to identify optimal approaches for LLM implementation, improve developer productivity, and accelerate development workflows.

LangChain also offers a full stack for AI application development:

- **Build**: with the composable framework
- **Run**: deploy with LangGraph Platform
- **Manage**: debug, test, and monitor with LangSmith

Based on our experience building with LangChain, here are some of its benefits we've found especially helpful:

- **Accelerated development cycles**: LangChain dramatically speeds up time-to-market with ready-made building blocks and unified APIs, eliminating weeks of integration work.
- **Superior observability**: The combination of LangChain and LangSmith provides unparalleled visibility into complex agent behavior, making trade-offs between cost, latency, and quality more transparent.
- **Controlled agency balance**: LangGraph's approach to agentic AI is particularly powerful—allowing developers to give LLMs partial control flow over workflows while maintaining reliability and performance.
- **Production-ready patterns**: Our implementation experience has proven that LangChain's architecture delivers enterprise-grade solutions that effectively reduce hallucinations and improve system reliability.
- **Future-proof flexibility**: The framework's vendor-agnostic design creates applications that can adapt as the LLM landscape evolves, preventing technological lock-in.

These advantages stem directly from LangChain's architectural decisions, which prioritize modularity, observability, and deployment flexibility for real-world applications.

Modular design and dependency management

LangChain evolves rapidly, with approximately 10-40 pull requests merged daily. This fast-paced development, combined with the framework's extensive integration ecosystem, presents unique challenges. Different integrations often require specific third-party Python packages, which can lead to dependency conflicts.

LangChain's package architecture evolved as a direct response to scaling challenges. As the framework rapidly expanded to support hundreds of integrations, the original monolithic structure became unsustainable—forcing users to install unnecessary dependencies, creating maintenance bottlenecks, and hindering contribution accessibility. By dividing into specialized packages with lazy loading of dependencies, LangChain elegantly solved these issues while preserving a cohesive ecosystem. This architecture allows developers to import only what they need, reduces version conflicts, enables independent release cycles for stable versus experimental features, and dramatically simplifies the contribution path for community developers working on specific integrations.

The LangChain codebase follows a well-organized structure that separates concerns while maintaining a cohesive ecosystem:

Core structure

- `docs/`: Documentation resources for developers
- `libs/`: Contains all library packages in the monorepo

Library organization

- `langchain-core/`: Foundational abstractions and interfaces that define the framework
- `langchain/`: The main implementation library with core components:
- `vectorstores/`: Integrations with vector databases (Pinecone, Chroma, etc.)
- `chains/`: Pre-built chain implementations for common workflows

Other component directories for retrievers, embeddings, etc.

- `langchain-experimental/`: Cutting-edge features still under development
- **langchain-community**: Houses third-party integrations maintained by the LangChain community. This includes most integrations for components like LLMs, vector stores, and retrievers. Dependencies are optional to maintain a lightweight package.
- **Partner packages**: Popular integrations are separated into dedicated packages (e.g., **langchain-openai**, **langchain-anthropic**) to enhance independent support. These packages reside outside the LangChain repository but within the GitHub "langchain-ai" organization (see `github.com/orgs/langchain-ai`). A full list is available at `python.langchain.com/v0.3/docs/integrations/platforms/`.

- **External partner packages**: Some partners maintain their integration packages independently. For example, several packages from the Google organization (`github.com/orgs/googleapis/repositories?q=langchain`), such as the `langchain-google-cloud-sql-mssql` package, are developed and maintained outside the LangChain ecosystem.

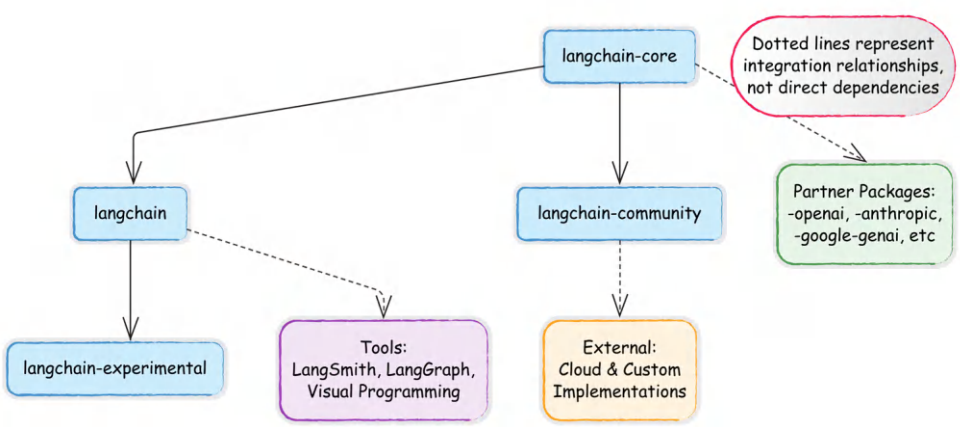

Figure 1.2: Integration ecosystem map

For full details on the dozens of available modules and packages, refer to the comprehensive LangChain API reference: `https://api.python.langchain.com/`. There are also hundreds of code examples demonstrating real-world use cases: `https://python.langchain.com/v0.1/docs/use_cases/`.

LangGraph, LangSmith, and companion tools

LangChain's core functionality is extended by the following companion projects:

- **LangGraph**: An orchestration framework for building stateful, multi-actor applications with LLMs. While it integrates smoothly with LangChain, it can also be used independently. LangGraph facilitates complex applications with cyclic data flows and supports streaming and human-in-the-loop interactions. We'll talk about LangGraph in more detail in *Chapter 3*.
- **LangSmith**: A platform that complements LangChain by providing robust debugging, testing, and monitoring capabilities. Developers can inspect, monitor, and evaluate their applications, ensuring continuous optimization and confident deployment.

These extensions, along with the core framework, provide a comprehensive ecosystem for developing, managing, and visualizing LLM applications, each with unique capabilities that enhance functionality and user experience.

LangChain also has an extensive array of tool integrations, which we'll discuss in detail in *Chapter 5*. New integrations are added regularly, expanding the framework's capabilities across domains.

Third-party applications and visual tools

Many third-party applications have been built on top of or around LangChain. For example, LangFlow and Flowise introduce visual interfaces for LLM development, with UIs that allow for the drag-and-drop assembly of LangChain components into executable workflows. This visual approach enables rapid prototyping and experimentation, lowering the barrier to entry for complex pipeline creation, as illustrated in the following screenshot of Flowise:

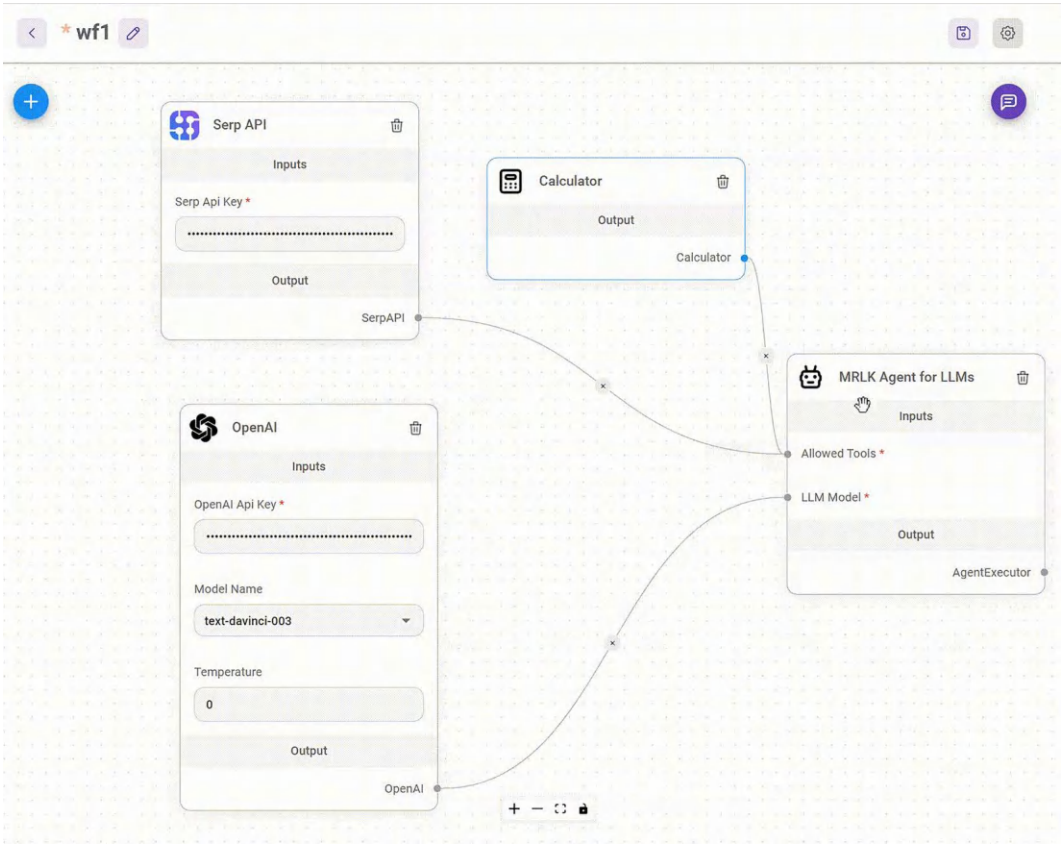

Figure 1.3: Flowise UI with an agent that uses an LLM, a calculator, and a search tool (Source: https://github.com/FlowiseAI/Flowise)

In the UI above, you can see an agent connected to a search interface (Serp API), an LLM, and a calculator. LangChain and similar tools can be deployed locally using libraries like Chainlit, or on various cloud platforms, including Google Cloud.

In summary, LangChain simplifies the development of LLM applications through its modular design, extensive integrations, and supportive ecosystem. This makes it an invaluable tool for developers looking to build sophisticated AI systems without reinventing fundamental components.

Summary

This chapter introduced the modern LLM landscape and positioned LangChain as a powerful framework for building production-ready AI applications. We explored the limitations of raw LLMs and then showed how these frameworks transform models into reliable, agentic systems capable of solving complex real-world problems. We also examined the LangChain ecosystem's architecture, including its modular components, package structure, and companion projects that support the complete development lifecycle. By understanding the relationship between LLMs and the frameworks that extend them, you're now equipped to build applications that go beyond simple text generation.

In the next chapter, we'll set up our development environment and take our first steps with LangChain, translating the conceptual understanding from this chapter into working code. You'll learn how to connect to various LLM providers, create your first chains, and begin implementing the patterns that form the foundation of enterprise-grade AI applications.

Questions

1. What are the three primary limitations of raw LLMs that impact production applications, and how does LangChain address each one?
2. Compare and contrast open-source and closed-source LLMs in terms of deployment options, cost considerations, and use cases. When might you choose each type?
3. What is the difference between a LangChain chain and a LangGraph agent? When would you choose one over the other?
4. Explain how LangChain's modular architecture supports the rapid development of AI applications. Provide an example of how this modularity might benefit an enterprise use case.
5. What are the key components of the LangChain ecosystem, and how do they work together to support the development lifecycle from building to deployment to monitoring?
6. How does agentic AI differ from traditional LLM applications? Describe a business scenario where an agent would provide significant advantages over a simple chain.

7. What factors should you consider when selecting an LLM provider for a production application? Name at least three considerations beyond just model performance.
8. How does LangChain help address common challenges like hallucinations, context limitations, and tool integration that affect all LLM applications?
9. Explain how the LangChain package structure (`langchain-core`, `langchain`, `langchain-community`) affects dependency management and integration options in your applications.
10. What role does LangSmith play in the development lifecycle of production LangChain applications?

2

First Steps with LangChain

In the previous chapter, we explored LLMs and introduced LangChain as a powerful framework for building LLM-powered applications. We discussed how LLMs have revolutionized natural language processing with their ability to understand context, generate human-like text, and perform complex reasoning. While these capabilities are impressive, we also examined their limitations—hallucinations, context constraints, and lack of up-to-date knowledge.

In this chapter, we'll move from theory to practice by building our first LangChain application. We'll start with the fundamentals: setting up a proper development environment, understanding LangChain's core components, and creating simple chains. From there, we'll explore more advanced capabilities, including running local models for privacy and cost efficiency and building multimodal applications that combine text with visual understanding. By the end of this chapter, you'll have a solid foundation in LangChain's building blocks and be ready to create increasingly sophisticated AI applications in subsequent chapters.

To sum up, this chapter will cover the following topics:

- Setting up dependencies
- Exploring LangChain's building blocks (model interfaces, prompts and templates, and LCEL)
- Running local models
- Multimodal AI applications

Given the rapid evolution of both LangChain and the broader AI field, we maintain up-to-date code examples and resources in our GitHub repository: `https://github.com/benman1/generative_ai_with_langchain`.

For questions or troubleshooting help, please create an issue on GitHub or join our Discord community: `https://packt.link/lang`.

Setting up dependencies for this book

This book provides multiple options for running the code examples, from zero-setup cloud notebooks to local development environments. Choose the approach that best fits your experience level and preferences. Even if you are familiar with dependency management, please read these instructions since all code in this book will depend on the correct installation of the environment as outlined here.

For the quickest start with no local setup required, we provide ready-to-use online notebooks for every chapter:

- **Google Colab**: Run examples with free GPU access
- **Kaggle Notebooks**: Experiment with integrated datasets
- **Gradient Notebooks**: Access higher-performance compute options

All code examples you find in this book are available as online notebooks on GitHub at `https://github.com/benman1/generative_ai_with_langchain`.

These notebooks don't have all dependencies pre-configured but, usually, a few install commands get you going. These tools allow you to start experimenting immediately without worrying about setup. If you prefer working locally, we recommend using conda for environment management:

1. Install Miniconda if you don't have it already.
2. Download it from `https://docs.conda.io/en/latest/miniconda.html`.
3. Create a new environment with Python 3.11:

```
conda create -n langchain-book python=3.11
```

4. Activate the environment:

```
conda activate langchain-book
```

5. Install Jupyter and core dependencies:

```
conda install jupyter
pip install langchain langchain-openai jupyter
```

6. Launch Jupyter Notebook:

```
jupyter notebook
```

This approach provides a clean, isolated environment for working with LangChain. For experienced developers with established workflows, we also support:

- **pip with venv**: Instructions in the GitHub repository
- **Docker containers**: Dockerfiles provided in the GitHub repository
- **Poetry**: Configuration files available in the GitHub repository

Choose the method you're most comfortable with but remember that all examples assume a Python 3.10+ environment with the dependencies listed in requirements.txt.

For developers, Docker, which provides isolation via containers, is a good option. The downside is that it uses a lot of disk space and is more complex than the other options. For data scientists, I'd recommend Conda or Poetry.

Conda handles intricate dependencies efficiently, although it can be excruciatingly slow in large environments. Poetry resolves dependencies well and manages environments; however, it doesn't capture system dependencies.

All tools allow sharing and replicating dependencies from configuration files. You can find a set of instructions and the corresponding configuration files in the book's repository at `https://github.com/benman1/generative_ai_with_langchain`.

Once you are finished, please make sure you have LangChain version 0.3.17 installed. You can check this with the command `pip show langchain`.

With the rapid pace of innovation in the LLM field, library updates are frequent. The code in this book is tested with LangChain 0.3.17, but newer versions may introduce changes. If you encounter any issues running the examples:

- Create an issue on our GitHub repository
- Join the discussion on Discord at `https://packt.link/lang`
- Check the errata on the book's Packt page

This community support ensures you'll be able to successfully implement all projects regardless of library updates.

API key setup

LangChain's provider-agnostic approach supports a wide range of LLM providers, each with unique strengths and characteristics. Unless you use a local LLM, to use these services, you'll need to obtain the appropriate authentication credentials.

Provider	Environment Variable	Setup URL	Free Tier?
OpenAI	`OPENAI_API_KEY`	`platform.openai.com`	No
HuggingFace	`HUGGINGFACEHUB_API_TOKEN`	`huggingface.co/settings/tokens`	Yes
Anthropic	`ANTHROPIC_API_KEY`	`console.anthropic.com`	No
Google AI	`GOOGLE_API_KEY`	`ai.google.dev/gemini-api`	Yes
Google VertexAI	`Application Default Credentials`	`cloud.google.com/vertex-ai`	Yes (with limits)
Replicate	`REPLICATE_API_TOKEN`	`replicate.com`	No

Table 2.1: API keys reference table (overview)

Most providers require an API key, while cloud providers like AWS and Google Cloud also support alternative authentication methods like **Application Default Credentials (ADC)**. Many providers offer free tiers without requiring credit card details, making it easy to get started.

To set an API key in an environment, in Python, we can execute the following lines:

```
import os
os.environ["OPENAI_API_KEY"] = "<your token>"
```

Here, `OPENAI_API_KEY` is the environment key that is appropriate for OpenAI. Setting the keys in your environment has the advantage of not needing to include them as parameters in your code every time you use a model or service integration.

You can also expose these variables in your system environment from your terminal. In Linux and macOS, you can set a system environment variable from the terminal using the `export` command:

```
export OPENAI_API_KEY=<your token>
```

To permanently set the environment variable in Linux or macOS, you would need to add the preceding line to the `~/.bashrc` or `~/.bash_profile` files, and then reload the shell using the command `source ~/.bashrc` or `source ~/.bash_profile`.

For Windows users, you can set the environment variable by searching for "Environment Variables" in the system settings, editing either "User variables" or "System variables," and adding `export OPENAI_API_KEY=your_key_here`.

Our choice is to create a `config.py` file where all API keys are stored. We then import a function from this module that loads these keys into the environment variables. This approach centralizes credential management and makes it easier to update keys when needed:

```
import os
OPENAI_API_KEY =  "... "
# I'm omitting all other keys

def set_environment():
    variable_dict = globals().items()
    for key, value in variable_dict:
        if "API" in key or "ID" in key:
             os.environ[key] = value
```

If you search for this file in the GitHub repository, you'll notice it's missing. This is intentional – I've excluded it from Git tracking using the `.gitignore` file. The `.gitignore` file tells Git which files to ignore when committing changes, which is essential for:

1. Preventing sensitive credentials from being publicly exposed
2. Avoiding accidental commits of personal API keys
3. Protecting yourself from unauthorized usage charges

To implement this yourself, simply add `config.py` to your `.gitignore` file:

```
# In .gitignore
config.py
.env
**/api_keys.txt
# Other sensitive files
```

You can set all your keys in the `config.py` file. This function, `set_environment()`, loads all the keys into the environment as mentioned. Anytime you want to run an application, you import the function and run it like so:

```
from config import set_environment
set_environment()
```

For production environments, consider using dedicated secrets management services or environment variables injected at runtime. These approaches provide additional security while maintaining the separation between code and credentials.

While OpenAI's models remain influential, the LLM ecosystem has rapidly diversified, offering developers multiple options for their applications. To maintain clarity, we'll separate LLMs from the model gateways that provide access to them.

- **Key LLM families**
    - **Anthropic Claude**: Excels in reasoning, long-form content processing, and vision analysis with up to 200K token context windows
    - **Mistral models**: Powerful open-source models with strong multilingual capabilities and exceptional reasoning abilities
    - **Google Gemini**: Advanced multimodal models with industry-leading 1M token context window and real-time information access
    - **OpenAI GPT-o**: Leading omnimodal capabilities accepting text, audio, image, and video with enhanced reasoning
    - **DeepSeek models:** Specialized in coding and technical reasoning with state-of-the-art performance on programming tasks
    - **AI21 Labs Jurassic:** Strong in academic applications and long-form content generation
    - **Inflection Pi**: Optimized for conversational AI with exceptional emotional intelligence
    - **Perplexity models**: Focused on accurate, cited answers for research applications
    - **Cohere models**: Specialized for enterprise applications with strong multilingual capabilities

- **Cloud provider gateways**
    - **Amazon Bedrock**: Unified API access to models from Anthropic, AI21, Cohere, Mistral, and others with AWS integration
    - **Azure OpenAI Service**: Enterprise-grade access to OpenAI and other models with robust security and Microsoft ecosystem integration
    - **Google Vertex AI**: Access to Gemini and other models with seamless Google Cloud integration
- **Independent platforms**
    - **Together AI**: Hosts 200+ open-source models with both serverless and dedicated GPU options
    - **Replicate**: Specializes in deploying multimodal open-source models with pay-as-you-go pricing
    - **HuggingFace Inference Endpoints**: Production deployment of thousands of open-source models with fine-tuning capabilities

Throughout this book, we'll work with various models accessed through different providers, giving you the flexibility to choose the best option for your specific needs and infrastructure requirements.

We will use OpenAI for many applications but will also try LLMs from other organizations. Refer to the *Appendix* at the end of the book to learn how to get API keys for OpenAI, Hugging Face, Google, and other providers.

There are two main integration packages:

- `langchain-google-vertexai`
- `langchain-google-genai`

We'll be using `langchain-google-genai`, the package recommended by LangChain for individual developers. The setup is a lot simpler, only requiring a Google account and API key. It is recommended to move to `langchain-google-vertexai` for larger projects. This integration offers enterprise features such as customer encryption keys, virtual private cloud integration, and more, requiring a Google Cloud account with billing.

If you've followed the instructions on GitHub, as indicated in the previous section, you should already have the `langchain-google-genai` package installed.

Exploring LangChain's building blocks

To build practical applications, we need to know how to work with different model providers. Let's explore the various options available, from cloud services to local deployments. We'll start with fundamental concepts like LLMs and chat models, then dive into prompts, chains, and memory systems.

Model interfaces

LangChain provides a unified interface for working with various LLM providers. This abstraction makes it easy to switch between different models while maintaining a consistent code structure. The following examples demonstrate how to implement LangChain's core components in practical scenarios.

Please note that users should almost exclusively be using the newer chat models as most model providers have adopted a chat-like interface for interacting with language models. We still provide the LLM interface, because it's very easy to use as string-in, string-out.

LLM interaction patterns

The LLM interface represents traditional text completion models that take a string input and return a string output. More and more use cases in LangChain use only the ChatModel interface, mainly because it's better suited for building complex workflows and developing agents. The LangChain documentation is now deprecating the LLM interface and recommending the use of chat-based interfaces. While this chapter demonstrates both interfaces, we recommend using chat models as they represent the current standard to be up to date with LangChain.

Let's see the LLM interface in action:

```
from langchain_openai import OpenAI
from langchain_google_genai import GoogleGenerativeAI

# Initialize OpenAI model
openai_llm = OpenAI()

# Initialize a Gemini model
gemini_pro = GoogleGenerativeAI(model="gemini-1.5-pro")
```

```
# Either one or both can be used with the same interface
response = openai_llm.invoke("Tell me a joke about light bulbs!")

print(response)
```

Please note that you must set your environment variables to the provider keys when you run this. For example, when running this I'd start the file by calling `set_environment() from config`:

```
from config import set_environment
set_environment()
```

We get this output:

```
Why did the light bulb go to therapy?
Because it was feeling a little dim!
```

For the Gemini model, we can run:

```
response = gemini_pro.invoke("Tell me a joke about light bulbs!")
```

For me, Gemini comes up with this joke:

```
Why did the light bulb get a speeding ticket?
Because it was caught going over the watt limit!
```

Notice how we use the same `invoke()` method regardless of the provider. This consistency makes it easy to experiment with different models or switch providers in production.

Development testing

During development, you might want to test your application without making actual API calls. LangChain provides `FakeListLLM` for this purpose:

```
from langchain_community.llms import FakeListLLM

# Create a fake LLM that always returns the same response
fake_llm = FakeListLLM(responses=["Hello"])

result = fake_llm.invoke("Any input will return Hello")
print(result)  # Output: Hello
```

Working with chat models

Chat models are LLMs that are fine-tuned for multi-turn interaction between a model and a human. These days most LLMs are fine-tuned for multi-turned conversations. Instead of providing input to the model, such as:

```
human: turn1
ai: answer1
human: turn2
ai: answer2
```

where we expect it to generate an output by continuing the conversation, these days model providers typically expose an API that expects each turn as a separate well-formatted part of the payload. Model providers typically don't store the chat history server-side, they get the full history sent each time from the client and only format the final prompt server-side.

LangChain follows the same pattern with ChatModels, processing conversations through structured messages with roles and content. Each message contains:

- Role (who's speaking), which is defined by the message class (all messages inherit from BaseMessage)
- Content (what's being said)

Message types include:

- `SystemMessage`: Sets behavior and context for the model. Example:

  ```
  SystemMessage(content="You're a helpful programming assistant")
  ```

- `HumanMessage`: Represents user input like questions, commands, and data. Example:

  ```
  HumanMessage(content="Write a Python function to calculate
  factorial")
  ```

- `AIMessage`: Contains model responses

Let's see this in action:

```
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import SystemMessage, HumanMessage

chat = ChatAnthropic(model="claude-3-opus-20240229")
messages = [
```

```
    SystemMessage(content="You're a helpful programming assistant"),
    HumanMessage(content="Write a Python function to calculate factorial")
]
response = chat.invoke(messages)
print(response)
```

Claude comes up with a function, an explanation, and examples for calling the function.

Here's a Python function that calculates the factorial of a given number:

```
```python
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    elif n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result
```
Let's break that down. The factorial function is designed to take an
integer n as input and calculate its factorial. It starts by checking if
n is negative, and if so, it raises a ValueError since factorials aren't
defined for negative numbers. If n is zero, the function returns 1, which
makes sense because, by definition, the factorial of 0 is 1.
When dealing with positive numbers, the function kicks things off by
setting a variable result to 1. From there, it enters a loop that runs
from 1 to n, inclusive, thanks to the range function. During each step
of the loop, it multiplies the result by the current number, gradually
building up the factorial. Once the loop completes, the function returns
the final calculated value. You can call this function by providing a non-
negative integer as an argument. Here are a few examples:
```python
print(factorial(0))  # Output: 1
print(factorial(5))  # Output: 120
print(factorial(10))  # Output: 3628800
print(factorial(-5))  # Raises ValueError: Factorial is not defined for
negative numbers.
```
```

```
Note that the factorial function grows very quickly, so calculating the
factorial of large numbers may exceed the maximum representable value in
Python. In such cases, you might need to use a different approach or a
library that supports arbitrary-precision arithmetic.
```

Similarly, we could have asked an OpenAI model such as GPT-4 or GPT-4o:

```
from langchain_openai.chat_models import ChatOpenAI
chat = ChatOpenAI(model_name='gpt-4o')
```

Reasoning models

Anthropic's Claude 3.7 Sonnet introduces a powerful capability called *extended thinking* that allows the model to show its reasoning process before delivering a final answer. This feature represents a significant advancement in how developers can leverage LLMs for complex reasoning tasks.

Here's how to configure extended thinking through the ChatAnthropic class:

```
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate

# Create a template
template = ChatPromptTemplate.from_messages([
    ("system", "You are an experienced programmer and mathematical
analyst."),
    ("user", "{problem}")
])
# Initialize Claude with extended thinking enabled
chat = ChatAnthropic(
    model_name="claude-3-7-sonnet-20240326",  # Use latest model version
    max_tokens=64_000,                        # Total response length
limit
    thinking={"type": "enabled", "budget_tokens": 15000},  # Allocate
tokens for thinking
)

# Create and run a chain
chain = template | chat

# Complex algorithmic problem
problem = """
```

```
Design an algorithm to find the kth largest element in an unsorted array
with the optimal time complexity. Analyze the time and space complexity
of your solution and explain why it's optimal.
"""
# Get response with thinking included
response = chat.invoke([HumanMessage(content=problem)])
print(response.content)
```

The response will include Claude's step-by-step reasoning about algorithm selection, complexity analysis, and optimization considerations before presenting its final solution. In the preceding example:

- Out of the 64,000-token maximum response length, up to 15,000 tokens can be used for Claude's thinking process.
- The remaining ~49,000 tokens are available for the final response.
- Claude doesn't always use the entire thinking budget—it uses what it needs for the specific task. If Claude runs out of thinking tokens, it will transition to its final answer.

While Claude offers explicit thinking configuration, you can achieve similar (though not identical) results with other providers through different techniques:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a problem-solving assistant."),
    ("user", "{problem}")
])

# Initialize with reasoning_effort parameter
chat = ChatOpenAI(
    model="o3-mini","
    reasoning_effort="high"  # Options: "low", "medium", "high"
)

chain = template | chat
response = chain.invoke({"problem": "Calculate the optimal strategy
for..."})
```

```
chat = ChatOpenAI(model="gpt-4o")
chain = template | chat
response = chain.invoke({"problem": "Calculate the optimal strategy
for..."})
```

The `reasoning_effort` parameter streamlines your workflow by eliminating the need for complex reasoning prompts, allows you to adjust performance by reducing effort when speed matters more than detailed analysis, and helps manage token consumption by controlling how much processing power goes toward reasoning processes.

DeepSeek models also offer explicit thinking configuration through the LangChain integration.

Controlling model behavior

Understanding how to control an LLM's behavior is crucial for tailoring its output to specific needs. Without careful parameter adjustments, the model might produce overly creative, inconsistent, or verbose responses that are unsuitable for practical applications. For instance, in customer service, you'd want consistent, factual answers, while in content generation, you might aim for more creative and promotional outputs.

LLMs offer several parameters that allow fine-grained control over generation behavior, though exact implementation may vary between providers. Let's explore the most important ones:

Parameter	Description	Typical Range	Best For
Temperature	Controls randomness in text generation	0.0-1.0 (OpenAI, Anthropic) 0.0-2.0 (Gemini)	Lower (0.0-0.3): Factual tasks, Q&A Higher (0.7+): Creative writing, brainstorming
Top-k	Limits token selection to k most probable tokens	1-100	Lower values (1-10): More focused outputs Higher values: More diverse completions
Top-p (Nucleus Sampling)	Considers tokens until cumulative probability reaches threshold	0.0-1.0	Lower values (0.5): More focused outputs Higher values (0.9): More exploratory responses

Max tokens	Limits maximum response length	Model-specific	Controlling costs and preventing verbose outputs
Presence/frequency penalties	Discourages repetition by penalizing tokens that have appeared	-2.0 to 2.0	Longer content generation where repetition is undesirable
Stop sequences	Tells model when to stop generating	Custom strings	Controlling exact ending points of generation

Table 2.2: Parameters offered by LLMs

These parameters work together to shape model output:

- **Temperature + Top-k/Top-p**: First, Top-k/Top-p filter the token distribution, and then temperature affects randomness within that filtered set
- **Penalties + Temperature**: Higher temperatures with low penalties can produce creative but potentially repetitive text

LangChain provides a consistent interface for setting these parameters across different LLM providers:

```
from langchain_openai import OpenAI

# For factual, consistent responses
factual_llm = OpenAI(temperature=0.1, max_tokens=256)

# For creative brainstorming
creative_llm = OpenAI(temperature=0.8, top_p=0.95, max_tokens=512)
```

A few provider-specific considerations to keep in mind are:

- **OpenAI**: Known for consistent behavior with temperature in the 0.0-1.0 range
- **Anthropic**: May need lower temperature settings to achieve similar creativity levels to other providers
- **Gemini**: Supports temperature up to 2.0, allowing for more extreme creativity at higher settings
- **Open-source models**: Often require different parameter combinations than commercial APIs

Choosing parameters for applications

For enterprise applications requiring consistency and accuracy, lower temperatures (0.0-0.3) combined with moderate top-p values (0.5-0.7) are typically preferred. For creative assistants or brainstorming tools, higher temperatures produce more diverse outputs, especially when paired with higher top-p values.

Remember that parameter tuning is often empirical – start with provider recommendations, then adjust based on your specific application needs and observed outputs.

Prompts and templates

Prompt engineering is a crucial skill for LLM application development, particularly in production environments. LangChain provides a robust system for managing prompts with features that address common development challenges:

- **Template systems** for dynamic prompt generation
- **Prompt management and versioning** for tracking changes
- **Few-shot example management** for improved model performance
- **Output parsing and validation** for reliable results

LangChain's prompt templates transform static text into dynamic prompts with variable substitution – compare these two approaches to see the key differences:

1. Static use – problematic at scale:

```
 def generate_prompt(question, context=None):
    if context:
        return f"Context information: {context}\n\nAnswer this
question concisely: {question}"

    return f"Answer this question concisely: {question}"
      # example use:
      prompt_text = generate_prompt("What is the capital of
France?")
```

2. PromptTemplate – production-ready:

```
from langchain_core.prompts import PromptTemplate
# Define once, reuse everywhere
question_template = PromptTemplate.from_template( "Answer this
question concisely: {question}" )
```

```
question_with_context_template = PromptTemplate.from_template(
"Context information: {context}\n\nAnswer this question concisely:
{question}" )
# Generate prompts by filling in variables
prompt_text = question_template.format(question="What is the capital
of France?")
```

Templates matter – here's why:

- **Consistency**: They standardize prompts across your application.
- **Maintainability**: They allow you to change the prompt structure in one place instead of throughout your codebase.
- **Readability**: They clearly separate template logic from business logic.
- **Testability**: It is easier to unit test prompt generation separately from LLM calls.

In production applications, you'll often need to manage dozens or hundreds of prompts. Templates provide a scalable way to organize this complexity.

Chat prompt templates

For chat models, we can create more structured prompts that incorporate different roles:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

template = ChatPromptTemplate.from_messages([
    ("system", "You are an English to French translator."),
    ("user", "Translate this to French: {text}")
])

chat = ChatOpenAI()
formatted_messages = template.format_messages(text="Hello, how are you?")
response = chat.invoke(formatted_messages)
print(response)
```

Let's start by looking at **LangChain Expression Language (LCEL)**, which provides a clean, intuitive way to build LLM applications.

LangChain Expression Language (LCEL)

LCEL represents a significant evolution in how we build LLM-powered applications with LangChain. Introduced in August 2023, LCEL is a declarative approach to constructing complex LLM workflows. Rather than focusing on *how* to execute each step, LCEL lets you define *what* you want to accomplish, allowing LangChain to handle the execution details behind the scenes.

At its core, LCEL serves as a minimalist code layer that makes it remarkably easy to connect different LangChain components. If you're familiar with Unix pipes or data processing libraries like pandas, you'll recognize the intuitive syntax: components are connected using the pipe operator (|) to create processing pipelines.

As we briefly introduced in *Chapter 1*, LangChain has always used the concept of a "chain" as its fundamental pattern for connecting components. Chains represent sequences of operations that transform inputs into outputs.

Originally, LangChain implemented this pattern through specific `Chain` classes like `LLMChain` and `ConversationChain`. While these legacy classes still exist, they've been deprecated in favor of the more flexible and powerful LCEL approach, which is built upon the Runnable interface.

The Runnable interface is the cornerstone of modern LangChain. A Runnable is any component that can process inputs and produce outputs in a standardized way. Every component built with LCEL adheres to this interface, which provides consistent methods including:

- `invoke()`: Processes a single input synchronously and returns an output
- `stream()`: Streams output as it's being generated
- `batch()`: Efficiently processes multiple inputs in parallel
- `ainvoke()`, `abatch()`, `astream()`: Asynchronous versions of the above methods

This standardization means any Runnable component—whether it's an LLM, a prompt template, a document retriever, or a custom function—can be connected to any other Runnable, creating a powerful composability system.

Every Runnable implements a consistent set of methods including:

- `invoke()`: Processes a single input synchronously and returns an output
- `stream()`: Streams output as it's being generated

This standardization is powerful because it means any Runnable component—whether it's an LLM, a prompt template, a document retriever, or a custom function—can be connected to any other Runnable. The consistency of this interface enables complex applications to be built from simpler building blocks.

LCEL offers several advantages that make it the preferred approach for building LangChain applications:

- **Rapid development**: The declarative syntax enables faster prototyping and iteration of complex chains.
- **Production-ready features**: LCEL provides built-in support for streaming, asynchronous execution, and parallel processing.
- **Improved readability**: The pipe syntax makes it easy to visualize data flow through your application.
- **Seamless ecosystem integration**: Applications built with LCEL automatically work with LangSmith for observability and LangServe for deployment.
- **Customizability**: Easily incorporate custom Python functions into your chains with RunnableLambda.
- **Runtime optimization**: LangChain can automatically optimize the execution of LCEL-defined chains.

LCEL truly shines when you need to build complex applications that combine multiple components in sophisticated workflows. In the next sections, we'll explore how to use LCEL to build real-world applications, starting with the basic building blocks and gradually incorporating more advanced patterns.

The pipe operator (|) serves as the cornerstone of LCEL, allowing you to chain components sequentially:

```
# 1. Basic sequential chain: Just prompt to LLM
basic_chain = prompt | llm | StrOutputParser()
```

Here, `StrOutputParser()` is a simple output parser that extracts the string response from an LLM. It takes the structured output from an LLM and converts it to a plain string, making it easier to work with. This parser is especially useful when you need just the text content without metadata.

Under the hood, LCEL uses Python's operator overloading to transform this expression into a RunnableSequence where each component's output flows into the next component's input. The pipe (|) is syntactic sugar that overrides the `__or__` hidden method, in other words, `A | B` is equivalent to `B.__or__(A)`.

The pipe syntax is equivalent to creating a `RunnableSequence` programmatically:

```
chain = RunnableSequence(first= prompt, middle=[llm], last= output_parser)
LCEL also supports adding transformations and custom functions:
with_transformation = prompt | llm | (lambda x: x.upper()) |
StrOutputParser()
```

For more complex workflows, you can incorporate branching logic:

```
decision_chain = prompt | llm | (lambda x: route_based_on_content(x)) | {
    "summarize": summarize_chain,
    "analyze": analyze_chain
}
```

Non-Runnable elements like functions and dictionaries are automatically converted to appropriate Runnable types:

```
# Function to Runnable
length_func = lambda x: len(x)
chain = prompt | length_func | output_parser

# Is converted to:
chain = prompt | RunnableLambda(length_func) | output_parser
```

The flexible, composable nature of LCEL will allow us to tackle real-world LLM application challenges with elegant, maintainable code.

Simple workflows with LCEL

As we've seen, LCEL provides a declarative syntax for composing LLM application components using the pipe operator. This approach dramatically simplifies workflow construction compared to traditional imperative code. Let's build a simple joke generator to see LCEL in action:

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

# Create components
prompt = PromptTemplate.from_template("Tell me a joke about {topic}")
llm = ChatOpenAI()
output_parser = StrOutputParser()
```

```
# Chain them together using LCEL
chain = prompt | llm | output_parser

#  Execute the workflow with a single call
result = chain.invoke({"topic": "programming"})
print(result)
```

This produces a programming joke:

```
Why don't programmers like nature?
It has too many bugs!
```

Without LCEL, the same workflow is equivalent to separate function calls with manual data passing:

```
formatted_prompt = prompt.invoke({"topic": "programming"})
llm_output = llm.invoke(formatted_prompt)
result = output_parser.invoke(llm_output)
```

As you can see, we have detached chain construction from its execution.

In production applications, this pattern becomes even more valuable when handling complex workflows with branching logic, error handling, or parallel processing – topics we'll explore in *Chapter 3*.

Complex chain example

While the simple joke generator demonstrated basic LCEL usage, real-world applications typically require more sophisticated data handling. Let's explore advanced patterns using a story generation and analysis example.

In this example, we'll build a multi-stage workflow that demonstrates how to:

1. Generate content with one LLM call
2. Feed that content into a second LLM call
3. Preserve and transform data throughout the chain

```
from langchain_core.prompts import PromptTemplate
from langchain_google_genai import GoogleGenerativeAI
from langchain_core.output_parsers import StrOutputParser

# Initialize the model
llm = GoogleGenerativeAI(model="gemini-1.5-pro")
```

```
# First chain generates a story
story_prompt = PromptTemplate.from_template("Write a short story about
{topic}")
story_chain = story_prompt | llm | StrOutputParser()

# Second chain analyzes the story
analysis_prompt = PromptTemplate.from_template(
    "Analyze the following story's mood:\n{story}"
)
analysis_chain = analysis_prompt | llm | StrOutputParser()
```

We can compose these two chains together. Our first simple approach pipes the story directly into the analysis chain:

```
# Combine chains
story_with_analysis = story_chain | analysis_chain

# Run the combined chain
story_analysis = story_with_analysis.invoke({"topic": "a rainy day"})
print("\nAnalysis:", story_analysis)
```

I get a long analysis. Here's how it starts:

```
Analysis: The mood of the story is predominantly **calm, peaceful, and
subtly romantic.** There's a sense of gentle melancholy brought on by the
rain and the quiet emptiness of the bookshop, but this is balanced by a
feeling of warmth and hope.
```

While this works, we've lost the original story in our result – we only get the analysis! In production applications, we typically want to preserve context throughout the chain:

```
from langchain_core.runnables import RunnablePassthrough
# Using RunnablePassthrough.assign to preserve data
enhanced_chain = RunnablePassthrough.assign(
    story=story_chain  # Add 'story' key with generated content
).assign(
    analysis=analysis_chain  # Add 'analysis' key with analysis of the
story
)
# Execute the chain
```

```
result = enhanced_chain.invoke({"topic": "a rainy day"})
print(result.keys())  # Output: dict_keys(['topic', 'story', 'analysis'])
# dict_keys(['topic', 'story', 'analysis'])
```

For more control over the output structure, we could also construct dictionaries manually:

```
from operator import itemgetter
# Alternative approach using dictionary construction
manual_chain = (
    RunnablePassthrough() |  # Pass through input
    {
        "story": story_chain,  # Add story result
        "topic": itemgetter("topic")  # Preserve original topic
    } |
    RunnablePassthrough().assign(  # Add analysis based on story
        analysis=analysis_chain
    )
)

result = manual_chain.invoke({"topic": "a rainy day"})
print(result.keys())  # Output: dict_keys(['story', 'topic', 'analysis'])
```

We can simplify this with dictionary conversion using a LCEL shorthand:

```
# Simplified dictionary construction
simple_dict_chain = story_chain | {"analysis": analysis_chain}
result = simple_dict_chain.invoke({"topic": "a rainy day"}) print(result.
keys()) # Output: dict_keys(['analysis', 'output'])
```

What makes these examples more complex than our simple joke generator?

- **Multiple LLM calls**: Rather than a single prompt → LLM → parser flow, we're chaining multiple LLM interactions
- **Data transformation**: Using tools like `RunnablePassthrough` and `itemgetter` to manage and transform data
- **Dictionary preservation**: Maintaining context throughout the chain rather than just passing single values
- **Structured outputs**: Creating structured output dictionaries rather than simple strings

These patterns are essential for production applications where you need to:

- Track the provenance of generated content
- Combine results from multiple operations
- Structure data for downstream processing or display
- Implement more sophisticated error handling

While LCEL handles many complex workflows elegantly, for state management and advanced branching logic, you'll want to explore LangGraph, which we'll cover in *Chapter 3*.

While our previous examples used cloud-based models like OpenAI and Google's Gemini, LangChain's LCEL and other functionality work seamlessly with local models as well. This flexibility allows you to choose the right deployment approach for your specific needs.

Running local models

When building LLM applications with LangChain, you need to decide where your models will run.

- Advantages of local models:
    - Complete data control and privacy
    - No API costs or usage limits
    - No internet dependency
    - Control over model parameters and fine-tuning
- Advantages of cloud models:
    - No hardware requirements or setup complexity
    - Access to the most powerful, state-of-the-art models
    - Elastic scaling without infrastructure management
    - Continuous model improvements without manual updates
- When to choose local models:
    - Applications with strict data privacy requirements
    - Development and testing environments
    - Edge or offline deployment scenarios
    - Cost-sensitive applications with predictable, high-volume usage

Let's start with one of the most developer-friendly options for running local models.

Getting started with Ollama

Ollama provides a developer-friendly way to run powerful open-source models locally. It provides a simple interface for downloading and running various open-source models. The `langchain-ollama` dependency should already be installed if you've followed the instructions in this chapter; however, let's go through them briefly anyway:

1. Install the LangChain Ollama integration:

```
pip install langchain-ollama
```

2. Then pull a model. From the command line, a terminal such as bash or the WindowsPowerShell, run:

```
ollama pull deepseek-r1:1.5b
```

3. Start the Ollama server:

```
ollama serve
```

Here's how to integrate Ollama with the LCEL patterns we've explored:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
# Initialize Ollama with your chosen model
local_llm = ChatOllama(
    model="deepseek-r1:1.5b",
    temperature=0,
)

# Create an LCEL chain using the local model
prompt = PromptTemplate.from_template("Explain {concept} in simple terms")
local_chain = prompt | local_llm | StrOutputParser()
# Use the chain with your local model
result = local_chain.invoke({"concept": "quantum computing"})
print(result)
```

This LCEL chain functions identically to our cloud-based examples, demonstrating LangChain's model-agnostic design.

Please note that since you are running a local model, you don't need to set up any keys. The answer is very long – although quite reasonable. You can run this yourself and see what answers you get.

Now that we've seen basic text generation, let's look at another integration. Hugging Face offers an approachable way to run models locally, with access to a vast ecosystem of pre-trained models.

Working with Hugging Face models locally

With Hugging Face, you can either run a model locally (HuggingFacePipeline) or on the Hugging Face Hub (HuggingFaceEndpoint). Here, we are talking about local runs, so we'll focus on `HuggingFacePipeline`. Here we go:

```
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

# Create a pipeline with a small model:
llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation",
    pipeline_kwargs=dict(
        max_new_tokens=512,
        do_sample=False,
        repetition_penalty=1.03,
    ),
)

chat_model = ChatHuggingFace(llm=llm)

# Use it like any other LangChain LLM
messages = [
    SystemMessage(content="You're a helpful assistant"),
    HumanMessage(
        content="Explain the concept of machine learning in simple terms"
    ),
]
ai_msg = chat_model.invoke(messages)
print(ai_msg.content)
```

This can take quite a while, especially the first time, since the model has to be downloaded first. We've omitted the model response for the sake of brevity.

LangChain supports running models locally through other integrations as well, for example:

- **llama.cpp:** This high-performance C++ implementation allows running LLaMA-based models efficiently on consumer hardware. While we won't cover the setup process in detail, LangChain provides straightforward integration with llama.cpp for both inference and fine-tuning.
- **GPT4All**: GPT4All offers lightweight models that can run on consumer hardware. LangChain's integration makes it easy to use these models as drop-in replacements for cloud-based LLMs in many applications.

As you begin working with local models, you'll want to optimize their performance and handle common challenges. Here are some essential tips and patterns that will help you get the most out of your local deployments with LangChain.

Tips for local models

When working with local models, keep these points in mind:

1. **Resource management**: Local models require careful configuration to balance performance and resource usage. The following example demonstrates how to configure an Ollama model for efficient operation:

```
#  Configure model with optimized memory and processing settings

from langchain_ollama import ChatOllama
llm = ChatOllama(
  model="mistral:q4_K_M", # 4-bit quantized model (smaller memory
footprint)
  num_gpu=1, # Number of GPUs to utilize (adjust based on hardware)
 num_thread=4 # Number of CPU threads for parallel processing
)
```

   Let's look at what each parameter does:

   - **model="mistral:q4_K_M"**: Specifies a 4-bit quantized version of the Mistral model. Quantization reduces the model size by representing weights with fewer bits, trading minimal precision for significant memory savings. For example:
     - Full precision model: ~8GB RAM required
     - 4-bit quantized model: ~2GB RAM required

- **num_gpu=1**: Allocates GPU resources. Options include:
  - 0: CPU-only mode (slower but works without a GPU)
  - 1: Uses a single GPU (appropriate for most desktop setups)
  - Higher values: For multi-GPU systems only
- **num_thread=4**: Controls CPU parallelization:
  - Lower values (2-4): Good for running alongside other applications
  - Higher values (8-16): Maximizes performance on dedicated servers
  - Optimal setting: Usually matches your CPU's physical core count

2. **Error handling**: Local models can encounter various errors, from out-of-memory conditions to unexpected terminations. A robust error-handling strategy is essential:

```
def safe_model_call(llm, prompt, max_retries=2):
    """Safely call a local model with retry logic and graceful
    failure"""
    retries = 0
    while retries <= max_retries:
        try:
            return llm.invoke(prompt)
        except RuntimeError as e:
            # Common error with local models when running out of VRAM
            if "CUDA out of memory" in str(e):
                print(f"GPU memory error, waiting and retrying
({retries+1}/{max_retries+1})")
                time.sleep(2)  # Give system time to free resources
                retries += 1
            else:
                print(f"Runtime error: {e}")
                return "An error occurred while processing your request."
        except Exception as e:
            print(f"Unexpected error calling model: {e}")
            return "An error occurred while processing your request."
    # If we exhausted retries
    return "Model is currently experiencing high load. Please try again
later."

# Use the safety wrapper in your LCEL chain
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.runnables import RunnableLambda
prompt = PromptTemplate.from_template("Explain {concept} in simple terms")
safe_llm = RunnableLambda(lambda x: safe_model_call(llm, x))
safe_chain = prompt | safe_llm
response = safe_chain.invoke({"concept": "quantum computing"})
```

Common local model errors you might run into are as follows:

- **Out of memory**: Occurs when the model requires more VRAM than available
- **Model loading failure**: When model files are corrupt or incompatible
- **Timeout issues**: When inference takes too long on resource-constrained systems
- **Context length errors**: When input exceeds the model's maximum token limit

By implementing these optimizations and error-handling strategies, you can create robust LangChain applications that leverage local models effectively while maintaining a good user experience even when issues arise.

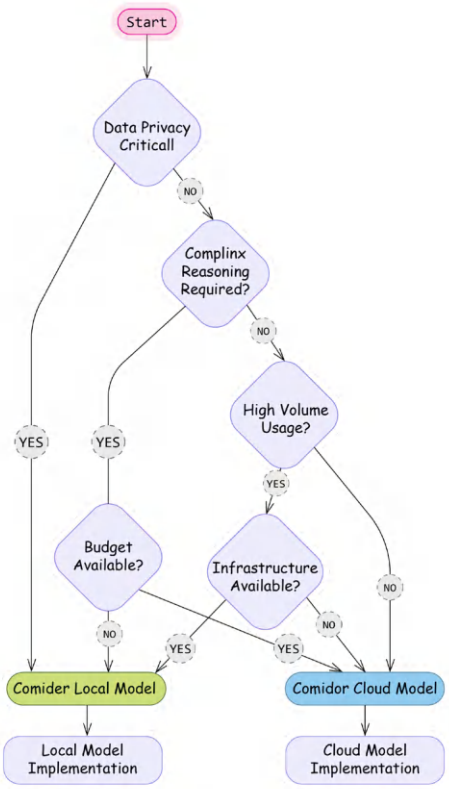

Figure 2.1: Decision chart for choosing between local and cloud-based models

Having explored how to build text-based applications with LangChain, we'll now extend our understanding to multimodal capabilities. As AI systems increasingly work with multiple forms of data, LangChain provides interfaces for both generating images from text and understanding visual content – capabilities that complement the text processing we've already covered and open new possibilities for more immersive applications.

Multimodal AI applications

AI systems have evolved beyond text-only processing to work with diverse data types. In the current landscape, we can distinguish between two key capabilities that are often confused but represent different technological approaches.

Multimodal understanding represents the ability of models to process multiple types of inputs simultaneously to perform reasoning and generate responses. These advanced systems can understand the relationships between different modalities, accepting inputs like text, images, PDFs, audio, video, and structured data. Their processing capabilities include cross-modal reasoning, context awareness, and sophisticated information extraction. Models like Gemini 2.5, GPT-4V, Sonnet 3.7, and Llama 4 exemplify this capability. For instance, a multimodal model can analyze a chart image along with a text question to provide insights about the data trend, combining visual and textual understanding in a single processing flow.

Content generation capabilities, by contrast, focus on creating specific types of media, often with extraordinary quality but more specialized functionality. Text-to-image models create visual content from descriptions, text-to-video systems generate video clips from prompts, text-to-audio tools produce music or speech, and image-to-image models transform existing visuals. Examples include Midjourney, DALL-E, and Stable Diffusion for images; Sora and Pika for video; and Suno and ElevenLabs for audio. Unlike true multimodal models, many generation systems are specialized for their specific output modality, even if they can accept multiple input types. They excel at creation rather than understanding.

As LLMs evolve beyond text, LangChain is expanding to support both multimodal understanding and content generation workflows. The framework provides developers with tools to incorporate these advanced capabilities into their applications without needing to implement complex integrations from scratch. Let's start with generating images from text descriptions. LangChain provides several approaches to incorporate image generation through external integrations and wrappers. We'll explore multiple implementation patterns, starting with the simplest and progressing to more sophisticated techniques that can be incorporated into your applications.

Text-to-image

LangChain integrates with various image generation models and services, allowing you to:

- Generate images from text descriptions
- Edit existing images based on text prompts
- Control image generation parameters
- Handle image variations and styles

LangChain includes wrappers and models for popular image generation services. First, let's see how to generate images with OpenAI's DALL-E model series.

Using DALL-E through OpenAI

LangChain's wrapper for DALL-E simplifies the process of generating images from text prompts. The implementation uses OpenAI's API under the hood but provides a standardized interface consistent with other LangChain components.

```
from langchain_community.utilities.dalle_image_generator import
DallEAPIWrapper

dalle = DallEAPIWrapper(
   model_name="dall-e-3",  # Options: "dall-e-2" (default) or "dall-e-3"
   size="1024x1024",       # Image dimensions
    quality="standard",     # "standard" or "hd" for DALL-E 3
    n=1                     # Number of images to generate (only for
DALL-E 2)
)

# Generate an image
image_url = dalle.run("A detailed technical diagram of a quantum
computer")

# Display the image in a notebook
from IPython.display import Image, display
display(Image(url=image_url))

# Or save it locally
import requests
response = requests.get(image_url)
```

```
with open("generated_library.png", "wb") as f:
    f.write(response.content)
```

Here's the image we got:

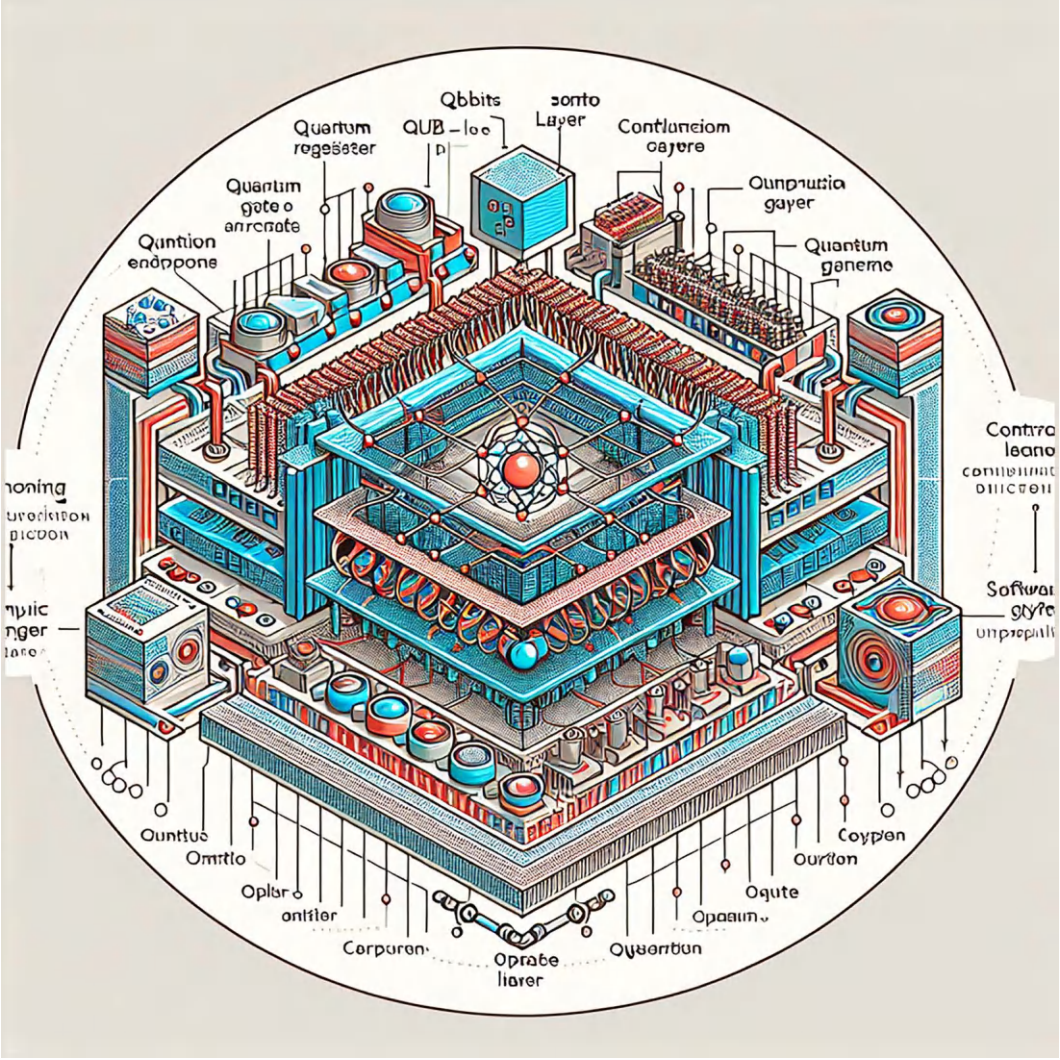

Figure 2.2: An image generated by OpenAI's DALL-E Image Generator

You might notice that text generation within these images is not one of the strong suites of these models. You can find a lot of models for image generation on Replicate, including the latest Stable Diffusion models, so this is what we'll use now.

Using Stable Diffusion

Stable Diffusion 3.5 Large is Stability AI's latest text-to-image model, released in March 2024. It's a **Multimodal Diffusion Transformer** (**MMDiT**) that generates high-resolution images with remarkable detail and quality.

This model uses three fixed, pre-trained text encoders and implements Query-Key Normalization for improved training stability. It's capable of producing diverse outputs from the same prompt and supports various artistic styles.

```
from langchain_community.llms import Replicate

# Initialize the text-to-image model with Stable Diffusion 3.5 Large
text2image = Replicate(
    model="stability-ai/stable-diffusion-3.5-large",
    model_kwargs={
        "prompt_strength": 0.85,
        "cfg": 4.5,
        "steps": 40,
        "aspect_ratio": "1:1",
        "output_format": "webp",
        "output_quality": 90
    }
)

# Generate an image
image_url = text2image.invoke(
    "A detailed technical diagram of an AI agent"
)
```

The recommended parameters for the new model include:

- **prompt_strength**: Controls how closely the image follows the prompt (0.85)
- **cfg**: Controls how strictly the model follows the prompt (4.5)
- **steps**: More steps result in higher-quality images (40)
- **aspect_ratio**: Set to 1:1 for square images
- **output_format**: Using WebP for a better quality-to-size ratio
- **output_quality**: Set to 90 for high-quality output

Here's the image we got:

Figure 2.3: An image generated by Stable Diffusion

Now let's explore how to analyze and understand images using multimodal models.

Image understanding

Image understanding refers to an AI system's ability to interpret and analyze visual information in ways similar to human visual perception. Unlike traditional computer vision (which focuses on specific tasks like object detection or facial recognition), modern multimodal models can perform general reasoning about images, understanding context, relationships, and even implicit meaning within visual content.

Gemini 2.5 Pro and GPT-4 Vision, among other models, can analyze images and provide detailed descriptions or answer questions about them.

Using Gemini 1.5 Pro

LangChain handles multimodal input through the same `ChatModel` interface. It accepts `Messages` as an input, and a `Message` object has a `content` field. IA `content` can consist of multiple parts, and each part can represent a different modality (that allows you to mix different modalities in your prompt).

You can send multimodal input by value or by reference. To send it by value, you should encode bytes as a string and construct an `image_url` variable formatted as in the example below using the image we generated using Stable Diffusion:

```
import base64
from langchain_google_genai.chat_models import ChatGoogleGenerativeAI
from langchain_core.messages.human import HumanMessage

with open("stable-diffusion.png", 'rb') as image_file:
    image_bytes = image_file.read()
    base64_bytes = base64.b64encode(image_bytes).decode("utf-8")

prompt = [
   {"type": "text", "text": "Describe the image: "},
   {"type": "image_url", "image_url": {"url": f"{base64_bytes}"}},
]

llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-pro",
    temperature=0,
)
response = llm.invoke([HumanMessage(content=prompt)])
print(response.content)
```

```
The image presents a futuristic, stylized depiction of a humanoid robot's
upper body against a backdrop of glowing blue digital displays. The robot's
head is rounded and predominantly white, with sections of dark, possibly
metallic, material around the face and ears.  The face itself features
glowing orange eyes and a smooth, minimalist design, lacking a nose or
mouth in the traditional human sense.  Small, bright dots, possibly LEDs
or sensors, are scattered across the head and body, suggesting advanced
technology and intricate construction.

The robot's neck and shoulders are visible, revealing a complex internal
structure of dark, interconnected parts, possibly wires or cables, which
contrast with the white exterior. The shoulders and upper chest are also
white, with similar glowing dots and hints of the internal mechanisms showing
through. The overall impression is of a sleek, sophisticated machine.
```

```
The background is a grid of various digital interfaces, displaying graphs,
charts, and other abstract data visualizations. These elements are all in
shades of blue, creating a cool, technological ambiance that complements
the robot's appearance. The displays vary in size and complexity, adding
to the sense of a sophisticated control panel or monitoring system. The
combination of the robot and the background suggests a theme of advanced
robotics, artificial intelligence, or data analysis.
```

As multimodal inputs typically have a large size, sending raw bytes as part of your request might not be the best idea. You can send it by reference by pointing to the blob storage, but the specific type of storage depends on the model's provider. For example, Gemini accepts multimedia input as a reference to Google Cloud Storage – a blob storage service provided by Google Cloud.

```
prompt = [
   {"type": "text", "text": "Describe the video in a few sentences."},
   {"type": "media", "file_uri": video_uri, "mime_type": "video/mp4"},
]

response = llm.invoke([HumanMessage(content=prompt)])
print(response.content)
```

Exact details on how to construct a multimodal input might depend on the provider of the LLM (and a corresponding LangChain integration handles a dictionary corresponding to a part of a `content` field accordingly). For example, Gemini accepts an additional `"video_metadata"` key that can point to the start and/or end offset of a video piece to be analyzed:

```
offset_hint = {
           "start_offset": {"seconds": 10},
           "end_offset": {"seconds": 20},
       }
prompt = [
   {"type": "text", "text": "Describe the video in a few sentences."},
   {"type": "media", "file_uri": video_uri, "mime_type": "video/mp4",
"video_metadata": offset_hint},
]

response = llm.invoke([HumanMessage(content=prompt)])
print(response.content)
```

And, of course, such multimodal parts can also be templated. Let's demonstrate it with a simple template that expects an `image_bytes_str` argument that contains encoded bytes:

```
prompt = ChatPromptTemplate.from_messages(
   [("user",
    [{"type": "image_url",
      "image_url": {"url": "{image_bytes_str}"},
      }])]
)
prompt.invoke({"image_bytes_str": "test-url"})
```

Using GPT-4 Vision

After having explored image generation, let's examine how LangChain handles image understanding using multimodal models. GPT-4 Vision capabilities (available in models like GPT-4o and GPT-4o-mini) allow us to analyze images alongside text, enabling applications that can "see" and reason about visual content.

LangChain simplifies working with these models by providing a consistent interface for multimodal inputs. Let's implement a flexible image analyzer:

```
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI

def analyze_image(image_url: str, question: str) -> str:
    chat = ChatOpenAI(model="gpt-4o-mini", max_tokens=256)

    message = HumanMessage(
        content=[
            {
                "type": "text",
                "text": question
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": image_url,
                    "detail": "auto"
                }
            }
```

```
        ]
    )

    response = chat.invoke([message])
    return response.content

# Example usage
image_url = "https://replicate.delivery/yhqm/
pMrKGpyPDip0LRciwSzrSOKb5ukcyXCyft0IBElxsT7fMrLUA/out-0.png"

questions = [
    "What objects do you see in this image?",
    "What is the overall mood or atmosphere?",
    "Are there any people in the image?"
]

for question in questions:
    print(f"\nQ: {question}")
    print(f"A: {analyze_image(image_url, question)}")
```

The model provides a rich, detailed analysis of our generated cityscape:

```
Q: What objects do you see in this image?
A: The image features a futuristic cityscape with tall, sleek skyscrapers.
The buildings appear to have a glowing or neon effect, suggesting a high-
tech environment. There is a large, bright sun or light source in the
sky, adding to the vibrant atmosphere. A road or pathway is visible in
the foreground, leading toward the city, possibly with light streaks
indicating motion or speed. Overall, the scene conveys a dynamic,
otherworldly urban landscape.
Q: What is the overall mood or atmosphere?
A: The overall mood or atmosphere of the scene is futuristic and vibrant.
The glowing outlines of the skyscrapers and the bright sunset create a
sense of energy and possibility. The combination of deep colors and light
adds a dramatic yet hopeful tone, suggesting a dynamic and evolving urban
environment.
Q: Are there any people in the image?
A: There are no people in the image. It appears to be a futuristic
cityscape with tall buildings and a sunset.
```

This capability opens numerous possibilities for LangChain applications. By combining image analysis with the text processing patterns we explored earlier in this chapter, you can build sophisticated applications that reason across modalities. In the next chapter, we'll build on these concepts to create more sophisticated multimodal applications.

Summary

After setting up our development environment and configuring necessary API keys, we've explored the foundations of LangChain development, from basic chains to multimodal capabilities. We've seen how LCEL simplifies complex workflows and how LangChain integrates with both text and image processing. These building blocks prepare us for more advanced applications in the coming chapters.

In the next chapter, we'll expand on these concepts to create more sophisticated multimodal applications with enhanced control flow, structured outputs, and advanced prompt techniques. You'll learn how to combine multiple modalities in complex chains, incorporate more sophisticated error handling, and build applications that leverage the full potential of modern LLMs.

Review questions

1. What are the three main limitations of raw LLMs that LangChain addresses?
    - Memory limitations
    - Tool integration
    - Context constraints
    - Processing speed
    - Cost optimization
2. Which of the following best describes the purpose of LCEL (LangChain Expression Language)?
    - A programming language for LLMs
    - A unified interface for composing LangChain components
    - A template system for prompts
    - A testing framework for LLMs
3. Name three types of memory systems available in LangChain
4. Compare and contrast LLMs and chat models in LangChain. How do their interfaces and use cases differ?

5. What role do Runnables play in LangChain? How do they contribute to building modular LLM applications?
6. When running models locally, which factors affect model performance? (Select all that apply)
    - Available RAM
    - CPU/GPU capabilities
    - Internet connection speed
    - Model quantization level
    - Operating system type
7. Compare the following model deployment options and identify scenarios where each would be most appropriate:
    - Cloud-based models (e.g., OpenAI)
    - Local models with llama.cpp
    - GPT4All integration
8. Design a basic chain using LCEL that would:
    - Take a user question about a product
    - Query a database for product information
    - Generate a response using an LLM
9. Provide a sketch outlining the components and how they connect.
10. Compare the following approaches for image analysis and mention the trade-offs between them:
    - Approach A

      ```
      from langchain_openai import ChatOpenAI
      chat = ChatOpenAI(model="gpt-4-vision-preview")
      ```

    - Approach B

      ```
      from langchain_community.llms import Ollama
      local_model = Ollama(model="llava")
      ```

Subscribe to our weekly newsletter

Subscribe to AI_Distilled, the go-to newsletter for AI professionals, researchers, and innovators, at `https://packt.link/Q5UyU`.

3

Building Workflows with LangGraph

So far, we've learned about LLMs, LangChain as a framework, and how to use LLMs with LangChain in a vanilla mode (just asking to generate a text output based on a prompt). In this chapter, we'll start with a quick introduction to LangGraph as a framework and how to develop more complex workflows with LangChain and LangGraph by chaining together multiple steps. As an example, we'll discuss parsing LLM outputs and look into error handling patterns with LangChain and LangGraph. Then, we'll continue with more advanced ways to develop prompts and explore what building blocks LangChain offers for few-shot prompting and other techniques.

We're also going to cover working with multimodal inputs, utilizing the long context, and adjusting your workloads to overcome limitations related to the context window size. Finally, we'll look into the basic mechanisms of managing memory with LangChain. Understanding these fundamental and key techniques will help us read LangGraph code, understand tutorials and code samples, and develop our own complex workflows. We'll, of course, discuss what LangGraph workflows are and will continue building on that skill in *Chapters 5* and *6*.

In a nutshell, we'll cover the following main topics in this chapter:

- LangGraph fundamentals
- Prompt engineering
- Working with short context windows
- Understanding memory mechanisms

As always, you can find all the code samples on our public GitHub repository as Jupyter notebooks: `https://github.com/benman1/generative_ai_with_langchain/tree/second_edition/chapter3`.

LangGraph fundamentals

LangGraph is a framework developed by LangChain (as a company) that helps control and orchestrate workflows. Why do we need another orchestration framework? Let's park this question until *Chapter 5*, where we'll touch on agents and agentic workflows, but for now, let us mention the flexibility of LangGraph as an orchestration framework and its robustness in handling complex scenarios.

Unlike many other frameworks, LangGraph allows cycles (most other orchestration frameworks operate only with directly acyclic graphs), supports streaming out of the box, and has many pre-built loops and components dedicated to generative AI applications (for example, human moderation). LangGraph also has a very rich API that allows you to have very granular control of your execution flow if needed. This is not fully covered in our book, but just keep in mind that you can always use a more low-level API if you need to.

A **Directed Acyclic Graph (DAG)** is a special type of graph in graph theory and computer science. Its edges (connections between nodes) have a direction, which means that the connection from node A to node B is different from the connection from node B to node A. It has no cycles. In other words, there is no path that starts at a node and returns to the same node by following the directed edges.

DAGs are often used as a model of workflows in data engineering, where nodes are tasks and edges are dependencies between these tasks. For example, an edge from node A to node B means that we need output from node A to execute node B.

For now, let's start with the basics. If you're new to this framework, we would also highly recommend a free online course on LangGraph that is available at `https://academy.langchain.com/` to deepen your understanding.

State management

State management is crucial in real-world AI applications. For example, in a customer service chatbot, the state might track information such as customer ID, conversation history, and outstanding issues. LangGraph's state management lets you maintain this context across a complex workflow of multiple AI components.

LangGraph allows you to develop and execute complex workflows called **graphs**. We will use the words *graph* and *workflow* interchangeably in this chapter. A graph consists of nodes and edges between them. Nodes are components of your workflow, and a workflow has a *state*. What is it? Firstly, a state makes your nodes aware of the current context by keeping track of the user input and previous computations. Secondly, a state allows you to persist your workflow execution at any point in time. Thirdly, a state makes your workflow truly interactive since a node can change the workflow's behavior by updating the state. For simplicity, think about a state as a Python dictionary. Nodes are Python functions that operate on this dictionary. They take a dictionary as input and return another dictionary that contains keys and values to be updated in the state of the workflow.

Let's understand that with a simple example. First, we need to define a state's schema:

```
from typing_extensions import TypedDict

class JobApplicationState(TypedDict):
   job_description: str
   is_suitable: bool
   application: str
```

A `TypedDict` is a Python type constructor that allows to define dictionaries with a predefined set of keys and each key can have its own type (as opposed to a `Dict[str, str]` construction).

LangGraph state's schema shouldn't necessarily be defined as a `TypedDict`; you can use data classes or Pydantic models too.

After we have defined a schema for a state, we can define our first simple workflow:

```
from langgraph.graph import StateGraph, START, END, Graph

def analyze_job_description(state):
   print("...Analyzing a provided job description ...")
   return {"is_suitable": len(state["job_description"]) > 100}

def generate_application(state):
   print("...generating application...")
   return {"application": "some_fake_application"}

builder = StateGraph(JobApplicationState)
builder.add_node("analyze_job_description", analyze_job_description)
builder.add_node("generate_application", generate_application)

builder.add_edge(START, "analyze_job_description")
builder.add_edge("analyze_job_description", "generate_application")
builder.add_edge("generate_application", END)

graph = builder.compile()
```

Here, we defined two Python functions that are components of our workflow. Then, we defined our workflow by providing a state's schema, adding nodes and edges between them. `add_node` is a convenient way to add a component to your graph (by providing its name and a corresponding Python function), and you can reference this name later when you define edges with `add_edge`. `START` and `END` are reserved built-in nodes that define the beginning and end of the workflow accordingly.

Let's take a look at our workflow by using a built-in visualization mechanism:

```
from IPython.display import Image, display
display(Image(graph.get_graph().draw_mermaid_png()))
```

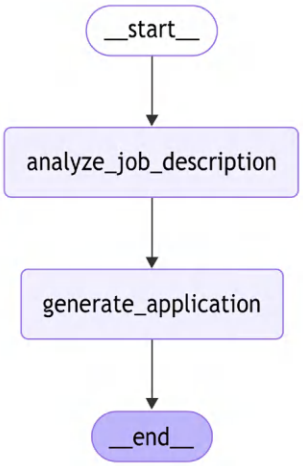

Figure 3.1: LangGraph built-in visualization of our first workflow

Our function accesses the state by simply reading from the dictionary that LangGraph automatically provides as input. LangGraph isolates state updates. When a node receives the state, it gets an immutable copy, not a reference to the actual state object. The node must return a dictionary containing the specific keys and values it wants to update. LangGraph then handles merging these updates into the master state. This pattern prevents side effects and ensures that state changes are explicit and traceable.

The only way for a node to modify a state is to provide an output dictionary with key-value pairs to be updated, and LangGraph will handle it. A node should modify at least one key in the state. A `graph` instance itself is a `Runnable` (to be precise, it inherits from `Runnable`) and we can execute it. We should provide a dictionary with the initial state, and we'll get the final state as an output:

```
res = graph.invoke({"job_description":"fake_jd"})
```

```
print(res)
>>...Analyzing a provided job description ...
...generating application...
{'job_description': 'fake_jd', 'is_suitable': True, 'application': 'some_
fake_application'}
```

We used a very simple graph as an example. With your real workflows, you can define parallel steps (for example, you can easily connect one node with multiple nodes) and even cycles. LangGraph executes the workflow in so-called *supersteps* that can call multiple nodes at the same time (and then merge state updates from these nodes). You can control the depth of recursion and amount of overall supersteps in the graph, which helps you avoid cycles running forever, especially because the LLMs output is non-deterministic.

A superstep on LangGraph represents a discrete iteration over one or a few nodes, and it's inspired by Pregel, a system built by Google for processing large graphs at scale. It handles parallel execution of nodes and updates sent to the central graph's state.

In our example, we used direct edges from one node to another. It makes our graph no different from a sequential chain that we could have defined with LangChain. One of the key LangGraph features is the ability to create conditional edges that can direct the execution flow to one or another node depending on the current state. A conditional edge is a Python function that gets the current state as an input and returns a string with the node's name to be executed.

Let's look at an example:

```
from typing import Literal

builder = StateGraph(JobApplicationState)
builder.add_node("analyze_job_description", analyze_job_description)
builder.add_node("generate_application", generate_application)

def is_suitable_condition(state: StateGraph) -> Literal["generate_
application", END]:
   if state.get("is_suitable"):
       return "generate_application"
   return END

builder.add_edge(START, "analyze_job_description")
builder.add_conditional_edges("analyze_job_description", is_suitable_
condition)
builder.add_edge("generate_application", END)

graph = builder.compile()
```

```
from IPython.display import Image, display
display(Image(graph.get_graph().draw_mermaid_png()))
```

We've defined an edge `is_suitable_condition` that takes a state and returns either an `END` or `generate_application` string by analyzing the current state. We used a `Literal` type hint since it's used by LangGraph to determine which destination nodes to connect the source node with when it's creating conditional edges. If you don't use a type hint, you can provide a list of destination nodes directly to the `add_conditional_edges` function; otherwise, LangGraph will connect the source node with all other nodes in the graph (since it doesn't analyze the code of an edge function itself when creating a graph). The following figure shows the output generated:

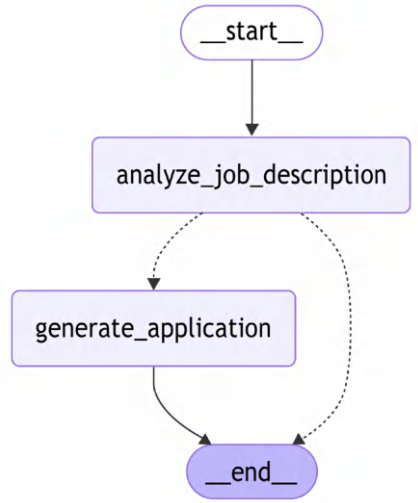

Figure 3.2: A workflow with conditional edges (represented as dotted lines)

Conditional edges are visualized with dotted lines, and now we can see that, depending on the output of the `analyze_job_description` step, our graph can perform different actions.

Reducers

So far, our nodes have changed the state by updating the value for a corresponding key. From another point of view, at each superstep, LangGraph can produce a new value for a given key. In other words, for every key in the state, there's a sequence of values, and from a functional programming perspective, a `reduce` function can be applied to this sequence. The default reducer on LangGraph always replaces the final value with the new value. Let's imagine we want to track custom actions (produced by nodes) and compare three options.

With the first option, a node should return a list as a value for the key `actions`. We provide short code samples just for illustration purposes, but you can find full ones on Github. If such a value already exists in the state, it will be replaced with the new one:

```
class JobApplicationState(TypedDict):
   ...
   actions: list[str]
```

Another option is to use the default `add` method with the `Annotated` type hint. By using this type hint, we tell the LangGraph compiler that the type of our variable in the state is a list of strings, and it should use the `add` method to concatenate two lists (if the value already exists in the state and a node produces a new one):

```
from typing import Annotated, Optional
from operator import add

class JobApplicationState(TypedDict):
   ...
   actions: Annotated[list[str], add]
```

The last option is to write your own custom reducer. In this example, we write a custom reducer that accepts not only a list from the node (as a new value) but also a single string that would be converted to a list:

```
from typing import Annotated, Optional, Union

def my_reducer(left: list[str], right: Optional[Union[str, list[str]]]) ->
list[str]:
 if right:
   return left + [right] if isinstance(right, str) else left + right
 return left

class JobApplicationState(TypedDict):
   ...
   actions: Annotated[list[str], my_reducer]
```

LangGraph has a few built-in reducers, and we'll also demonstrate how you can implement your own. One of the important ones is `add_messages`, which allows us to merge messages. Many of your nodes would be LLM agents, and LLMs typically work with messages. Therefore, according to the conversational programming paradigm we'll talk about in more detail in *Chapters 5* and *6*, you typically need to keep track of these messages:

```
from langchain_core.messages import AnyMessage
from langgraph.graph.message import add_messages

class JobApplicationState(TypedDict):
  ...
  messages: Annotated[list[AnyMessage], add_messages]
```

Since this is such an important reducer, there's a built-in state that you can inherit from:

```
from langgraph.graph import MessagesState

class JobApplicationState(MessagesState):
  ...
```

Now, as we have discussed reducers, let's talk about another important concept for any developer – how to write reusable and modular workflows by passing configurations to them.

Making graphs configurable

LangGraph provides a powerful API that allows you to make your graph configurable. It allows you to separate parameters from user input – for example, to experiment between different LLM providers or pass custom callbacks. A node can also access the configuration by accepting it as a second argument. The configuration will be passed as an instance of `RunnableConfig`.

`RunnableConfig` is a typed dictionary that gives you control over execution control settings. For example, you can control the maximum number of supersteps with the `recursion_limit` parameter. `RunnableConfig` also allows you to pass custom parameters as a separate dictionary under a `configurable` key.

Let's allow our node to use different LLMs during application generation:

```
from langchain_core.runnables.config import RunnableConfig

def generate_application(state: JobApplicationState, config:
RunnableConfig):
   model_provider = config["configurable"].get("model_provider", "Google")
   model_name = config["configurable"].get("model_name", "gemini-1.5-
flash-002")
   print(f"...generating application with {model_provider} and {model_
name} ...")
   return {"application": "some_fake_application", "actions": ["action2",
"action3"]}
```